

Streaming Function Secret Sharing and Its Applications

Xiangfu Song¹, Jianli Bai^{2,✉}, Ye Dong^{3,✉}, Yijian Liu^{4,5}, Yu Zhang^{4,5}, Xianhui Lu^{4,5}, Tianwei Zhang¹

¹Nanyang Technological University ²Singapore Management University

³National University of Singapore

⁴Institute of Information Engineering, Chinese Academy of Sciences

⁵School of Cyber Security, University of Chinese Academy of Sciences

Emails: {xiangfu.song,tianwei.zhang}@ntu.edu.sg, baijianli0812@gmail.com, dongye@nus.edu.sg,

{liuyijian,zhangyu1999,luxianhui}@iie.ac.cn, ✉ corresponding author

Abstract

Collecting statistics from users of software and online services is crucial to improve service quality, yet obtaining such insights while preserving individual privacy remains a challenge. Function secret sharing (FSS) is a promising tool for this problem. However, FSS-based solutions still face several challenges for streaming analytics, where messages are continuously sent, and secure computation tasks are repeatedly performed over incoming messages. We introduce a new cryptographic primitive called streaming function secret sharing (SFSS), a new variant of FSS that is particularly suitable for secure computation over streaming messages. We formalize SFSS and propose concrete constructions, including SFSS for point functions, predicate functions, and feasibility results for generic functions. SFSS powers several promising applications in a simple and modular fashion, including conditional transciphering, policy-hiding aggregation, and attribute-hiding aggregation. In particular, our SFSS formalization and constructions identify security flaws and efficiency bottlenecks in existing solutions, and SFSS-powered solutions achieve the expected security goal with asymptotically and concretely better efficiency and/or enhanced functionality.

1 Introduction

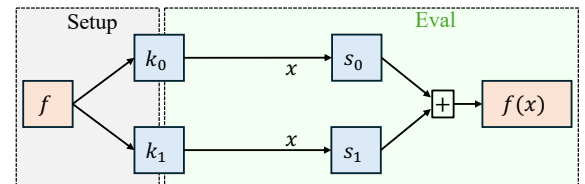
A (two-party) function secret sharing (FSS) [17, 18] secretly shares a function via a pair of keys, and any single key reveals no more information about the shared function. Concrete FSS constructions include distributed point function (DPF) [17, 18, 38], distributed comparison function (DCF) [17, 18, 19], and distributed multi-point function (DMPF) [16]. FSS has found applications in many secure computation protocols and applications [11, 12, 13, 19, 24, 28, 29, 49]. In particular, FSS is one of the main tools to build privacy-preserving measurement (PPM) systems [9, 28] in the distributed-trust model, which is deployed by Mozilla [42], Divvi Up [1], and Apple and Google [4], and is under standardization at IETF [37].

When applying FSS to applications, commonly, data is first encoded as a function of a certain family. Then FSS

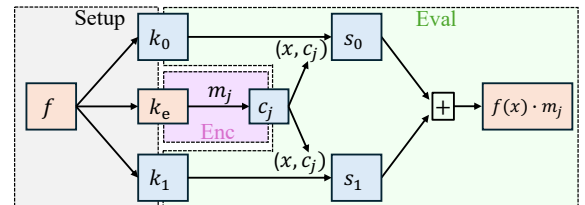
for that family is used to encode the function via a pair of FSS keys, which are distributed between two servers for the subsequent secure computation. Many applications, such as IoT services, healthcare monitoring, and federated learning, are naturally designed in the *streaming* setting: messages are continuously sent to the servers and secure computation is repeatedly performed over incoming messages. In this setting, simply encoding each new incoming message requires a new pair of FSS keys. Since the size of FSS keys is concretely high, when most portions of the streaming messages remain static, naïvely encoding data via new FSS keys incurs redundant overhead. We provide a convincing example in §2.1. This motivates a new primitive that is better suitable for secure computation over streaming data.

1.1 Formal Definition & Constructions

We initiate a new primitive called streaming function secret sharing (SFSS), capturing core efficiency and security properties for *reusable* secure computation over streaming messages.



(a) The sketched FSS workflow



(b) The sketched SFSS workflow

Figure 1: Comparison of FSS and SFSS in a nutshell.

SFSS definition. Fig. 1 sketches the difference between FSS and SFSS in a nutshell. A two-party FSS scheme consists of a Setup algorithm and an Eval algorithm. The Setup algorithm takes as input a function f , and outputs a pair of FSS keys (k_0, k_1) . Each FSS key k_b ($b \in \{0, 1\}$), when evaluated over an input x by Eval, outputs a share s_b such that $s_0 + s_1 = f(x)$. Namely, the local evaluation essentially shares $f(x)$.

SFSS captures how to decouple the (shared) function from a data stream to support key reuse, as well as efficiency properties such as *non-interactive* streaming encryption and *non-interactive* evaluation. It differs from FSS mainly in the following aspects: (1) The SFSS Setup algorithm additionally outputs a secret encryption key k_e besides the SFSS key pair. (2) SFSS introduces a streaming encryption algorithm Enc. Enc takes as input k_e and message m_j , outputs a streaming ciphertext c_j ; here j is a counter to identify the j -th streaming message and ciphertext. (3) The SFSS key k_b is evaluated over x and c_j , and outputs a share s_b such that $s_0 + s_1 = f(x) \cdot m_j$.

We stress that the SFSS Enc algorithm can be invoked possibly many times, and security requires that any single SFSS key k_b and all streaming ciphertexts are pseudorandom, meaning that $(k_b, \{c_j\}_j)$ reveals no more information about the function f or messages $\{m_j\}_j$.

SFSS constructions. We develop SFSS constructions for a broad class of functions. Overall, we “compile” FSS to SFSS.

1) *SFSS for single-point functions.* We start with SDPF for single-point functions; note that single-point functions are the most commonly used for FSS-based PPM. Our SDPF construction efficiently compiles any DPF scheme to its streaming version. The transformation only additionally requires a pseudorandom function (PRF). We design a conceptually simple and efficient compiler by 1) incorporating *pseudorandom shares* from DPF outputs as PRF keys, and 2) designing a new streaming masking and unmasking mechanism that introduces negligible overhead. We start with $\beta = 1$ and extend to the weighted case $\beta \geq 1$.

2) *SFSS for predicate functions.* We extend SFSS beyond single-point functions for general predicate functions; a predicate function only outputs binary values over all inputs. Unfortunately, the previous SDPF compiler fails to work if the function has multiple non-zero outputs. To overcome this, we propose a new compiler that relies on a *key-homomorphic pseudorandom function* (KH-PRF) [10] with a *special zero-key* property, which is not generally satisfied by all KH-PRFs but can be instantiated from the KH-PRF based on the *Learning-With-Rounding* (LWR) assumption [5].

3) *SFSS for generic functions.* We further study how SFSS supports generic functions. We introduce a *decomposition* technique that converts any function to its predicate form, and then apply SFSS for predicate functions again. This provides a guiding solution for functions with weighted outputs.

1.2 Applications

SFSS provides a modular approach for privacy-preserving applications over streaming data. Our formalization also helps us identify security/efficiency flaws in existing solutions.

Policy-hiding aggregation. We can use SFSS for designing *policy-hiding aggregation* (phAgg) [24]. In phAgg, each client attaches a secret policy to its data stream. The policy defines under which conditions the stream can be used for aggregation. Beyond hiding the data stream, phAgg additionally protects: 1) the policy itself, and 2) whether a data stream is involved in an aggregation task (*i.e.*, data stream access patterns).

SFSS provides a modular design for phAgg, by simply configuring the policy function in its setup phase. Our study reveals that Vizard [24], based on equality testing using DPF, suffers from security and efficiency flaws. In contrast, our SDPF-based solution securely realizes the functionality that Vizard failed to achieve, meanwhile offering strictly improved efficiency (even compared with the flawed Vizard). Moreover, by configuring SFSS with predicate functions or generic functions, the SFSS-based solution extends beyond equality-based policies, supporting a richer class of policy functions, highlighting the versatility of our formalization.

Attribute-hiding aggregation. We introduce *attribute-hiding aggregation* (ahAgg), in which each client attaches attributes to its data stream, and an aggregation task specifies a filtering policy that selects which streams should contribute to the aggregation. Indeed, ahAgg is the *dual* definition of phAgg. The goal is to aggregate over the selected streams while hiding which streams are involved, thereby preserving anonymity and preventing leakage through access patterns. This approach is particularly suited for aggregation tasks grouped by client attributes (*e.g.*, age or region), where data stream, attributes, and attribute membership need protection. We show that SDPF suffices for practical ahAgg instantiations and propose a non-interactive query mechanism for arbitrary filtering functions.

Window-based aggregation. We further show how SFSS-based metadata-hiding aggregation supports efficient window-based aggregation with only two calls of SFSS evaluation per data stream, while existing native FSS-based approaches perform FSS evaluation calls that is linear in the window size. This optimization demonstrates how to further reduce computational overhead when SFSS is used in concrete applications.

Conditional transciphering. SFSS provides a new concept called *conditional transciphering* (CT). CT converts a ciphertext to an additive sharing of its encrypted message only if certain conditions are met; 0 is shared otherwise. We note that CT is promising for secure streaming computation beyond metadata-hiding aggregation. More details defer to §5.2.

1.3 Implementation

Besides these promising asymptotic efficiency results, we implement SFSS and report concrete performance. For basic

primitives, we evaluate SFSS for single-point functions and comparison functions (as a representative predicate function). Our results confirm that SDPF introduces negligible overhead compared to DPF, while SFSS for predicate functions is more expensive due to the relatively expensive KH-PRF. Nevertheless, this cost is acceptable in practice, which can be further atomized for window-based aggregation. For SFSS applications, our policy-hiding aggregation achieves even better efficiency properties than Vizard, while significantly outperforming the DPF-based baseline. For example, the DPF-based baseline incurs a total of 569.34MB of client-server communication for 100 data streams with an average of 10K messages, compared to only 13.28MB for our SDPF-based solution.

1.4 Contributions

We summarize our main contributions as follows.

- We propose streaming function secret sharing, a new cryptographic primitive with a formal definition and concrete constructions. We study feasibility results on SFSS over specialized and generic functions. This is the first systematic study of the streaming notion of FSS.
- SFSS facilitates promising streaming applications, with connections to real-world applications and standardization. SFSS-powered applications are conceptually simple and modular, with improved security, efficiency, and/or enhanced functionality compared with existing approaches.
- We develop a range of techniques and optimizations by carefully exploiting properties of underlying primitives, some of which may be of independent interest.
- We have implemented our schemes and open-sourced the code. We expect our implementation would facilitate research and applications in the related fields.

Connection to IETF PPM standardization. IETF PPM WG is actively standardizing PPM [37]. SFSS provides a promising extension for FSS-based PPM to the streaming setting. We will report our results to IETF PPM WG.

Organization. §2 presents motivation, SFSS definition, and system/threat model for related applications. §3 defines useful notations and definitions. §4 gives concrete SFSS constructions. §5 presents SFSS applications. §6 reports performance. We provide related works in §7 and conclude in §8.

2 Problem Statement

2.1 Example: Weighted Histogram Collection

To concretely emphasize why SFSS is useful, we start by reviewing the DPF-based histogram collection in the two-server model [28]. Let h be the histogram domain size and N_c the number of clients. Each client i holds an input $x^{(i)} \in [h]$ and encodes it as a single-point function $f_{x^{(i)},1}$; $f_{x^{(i)},1}$ outputs

1 at $x^{(i)}$ and 0 elsewhere, and distributes the corresponding DPF keys between two non-colluded servers. To compute the histogram value at position $p \in [h]$, the servers evaluate the DPF keys and jointly compute $\sum_{i \in [N_c]} f_{x^{(i)},1}(p)$ in a secret-shared fashion. The final result is obtained by combining the server outputs. This technique ensures input privacy while enabling efficient, non-interactive server-side computation.

Challenges in the streaming setting. The prior approach does not scale well to streaming analytics, where clients contribute data across multiple rounds. Assume each input $x^{(i)}$ is a static attribute i (e.g., region or device type). In round j , client i submits a new message $m_j^{(i)}$. The goal is to compute a weighted histogram:

$$\sum_{i \in [N_c]} f_{x^{(i)},1}(p) \cdot m_j^{(i)}, \forall p \in [h].$$

A straightforward solution is to generate a fresh DPF for $f_{x^{(i)},m_j^{(i)}}$ in each round, where $f_{x^{(i)},m_j^{(i)}}$ outputs $m_j^{(i)}$ at $x^{(i)}$ and zero elsewhere. This requires each client to regenerate and distribute a new pair of DPF keys per round, even though $x^{(i)}$ remains unchanged, and each client only wants to send one streaming message of constant size. Note that each DPF key is of size $O(\lambda \log h)$ [17, 18, 39], which is still concretely high for large h . Regenerating and resending new DPF keys introduce substantial computation and communication overhead, especially in high-frequency streaming environments.

Streaming function secret sharing. To reduce this overhead, we expect a streaming variant of DPF. Ideally, each client performs a one-time setup by distributing a pair of *streaming DPF keys*. In each round j , the client simply sends a *constant-size* streaming ciphertext $c_j^{(i)}$, encrypting the new message $m_j^{(i)}$. Given the reusable streaming DPF keys and the ciphertext $c_j^{(i)}$, the servers can *non-interactively* compute additive sharing of $f_{x^{(i)},1}(p) \cdot m_j^{(i)}$ for any $p \in [h]$. This leads to an efficient protocol for *weighted* histogram aggregation with *optimal* per-message cost. This is what we want for a streaming DPF scheme, and SFSS generalizes the concept.

2.2 Formal Definition

Def. 1 presents the formal SFSS definition. First, the key generation algorithm Gen outputs a pair of SFSS keys and a streaming encryption key k_e . Second, SFSS introduces an encoding algorithm Enc that encrypts streaming messages, decoupling function setup from data availability. Third, the evaluation algorithm Eval takes a streaming ciphertext as input, enabling key reuse across multiple streaming messages.

Definition 1 (Streaming Function Secret Sharing). A *two-party streaming FSS (SFSS) scheme* $\Pi_{\text{SFSS}} = (\text{Gen}, \text{Enc}, \text{Eval})$ for a function family $\mathcal{F}$ is defined with the following syntax.

- $(st_f, k_e, k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f})$. On input 1^λ and a description $\hat{f} \in \{0, 1\}^*$ for $f : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}} \in \mathcal{F}$, output a key pair (k_0, k_1) , a streaming encryption key k_e , and a state st_f .
- $(st_f, c) \leftarrow \text{Enc}(st_f, k_e, m)$. On input the state st_f , the streaming encryption key k_e , and a message $m \in \mathbb{G}_{\text{out}}$, output a ciphertext $c \in \mathbb{G}_{\text{out}}$ and an (updated) state st_f .
- $s_b \leftarrow \text{Eval}(b, k_b, x, c)$. On input the party identifier $b \in \{0, 1\}$, the party's key k_b , an evaluation input $x \in \mathbb{G}_{\text{in}}$, and a streaming ciphertext $c \in \mathbb{G}_{\text{out}}$, output a share $s_b \in \mathbb{G}_{\text{out}}$.

We can define $st_f = \perp$ for a stateless definition. Thus, this definition captures both state-based and stateless constructions.

B-bounded correctness. For correctness, the ideal goal for evaluation is to share $m \cdot f(x)$. We define a more general correctness notion called B -bounded correctness, allowing the result to deviate from $m \cdot f(x)$ by at most B . Formally, an SFSS is correct if there exists $B \in \mathbb{G}_{\text{out}}$ such that for any function $f \in \mathcal{F}$ with description $\hat{f}$, any $x \in \mathbb{G}_{\text{in}}$, and any $m \in \mathbb{G}_{\text{out}}$,

$$\Pr \left[\begin{array}{l} (st_f, k_e, k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f}), \\ (st_f, c) \leftarrow \text{Enc}(st_f, k_e, m) : \\ \sum_{b \in \{0, 1\}} \text{Eval}(b, k_b, x, c) = m \cdot f(x) + e \end{array} \right] = 1$$

for some error $e \in [0, B]$. SFSS is *perfectly correct* if $B = 0$.

Security. SFSS requires that any single SFSS key, together with a polynomial many streaming ciphertexts, reveals no more information than permitted by the leakage function Leak , formally captured by a simulation-based definition:

Real $_{f,b}^{\mathcal{A}}(1^\lambda)$:

-
- 1: $(st_f, k_e, k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f})$
 - 2: $st_{\mathcal{A}} \leftarrow \mathcal{A}_0(1^\lambda, st_f, k_b)$
 - 3: **for** $j \in [1, q]$:
 - 4: $(st_{\mathcal{A}}, m_j) \leftarrow \mathcal{A}_j(1^\lambda, st_{\mathcal{A}}, st_f, k_b, (m_1, \dots, m_{j-1}), (c_1, \dots, c_{j-1}))$
 - 5: $(st_f, c_j) \leftarrow \text{Enc}(st_f, k_e, m_j)$
 - 6: **return** output $= (st_{\mathcal{A}}, k_b, (c_1, \dots, c_q))$

Ideal $_{f,b}^{\mathcal{S}}(1^\lambda)$:

-
- 1: $(st_S, st_f, k_b) \leftarrow \mathcal{S}_0(1^\lambda, b, \text{Leak}(\hat{f}))$
 - 2: $st_{\mathcal{A}} \leftarrow \mathcal{A}_0(1^\lambda, st_f, k_b)$
 - 3: **for** $j \in [1, q]$:
 - 4: $(st_{\mathcal{A}}, m_j) \leftarrow \mathcal{A}_j(1^\lambda, st_{\mathcal{A}}, st_f, k_b, (m_1, \dots, m_{j-1}), (c_1, \dots, c_{j-1}))$
 - 5: $(st_S, (st_f, c_j)) \leftarrow \mathcal{S}_j(1^\lambda, b, st_S, \text{Leak}(\hat{f}, (m_1, \dots, m_j)))$
 - 6: **return** output $= (st_{\mathcal{A}}, k_b, (c_1, \dots, c_q))$

SFSS is *adaptively secure* if for any function $f \in \mathcal{F}$, any $b \in \{0, 1\}$, and any probabilistic-polynomial-time (PPT) adversary $\mathcal{A} = (\mathcal{A}_0, \dots, \mathcal{A}_q)$ with $q = \text{poly}(\lambda)$, there exists a PPT simulator $\mathcal{S} = (\mathcal{S}_0, \dots, \mathcal{S}_q)$, s.t. for any PPT distinguisher $\mathcal{D}$:

$$\left| \Pr \left[\text{output} \leftarrow \text{Real}_{f,b}^{\mathcal{A}}(1^\lambda) : \mathcal{D}(1^\lambda, \text{output}) = 1 \right] - \Pr \left[\text{output} \leftarrow \text{Ideal}_{f,b}^{\mathcal{S}}(1^\lambda) : \mathcal{D}(1^\lambda, \text{output}) = 1 \right] \right| \leq \text{negl}(\lambda),$$

where the probabilities are over the coins of Gen and Enc , and $\text{Leak}(\hat{f}, m_1, \dots, m_j)$ reveals the function family $\mathcal{F}$, $\mathbb{G}_{\text{in}}$, $\mathbb{G}_{\text{out}}$, size j , and message/ciphertext length $|\mathbb{G}_{\text{out}}|$.

2.3 Applications: System and Threat Model

System model. Same as previous FSS-based privacy-preserving systems [28, 29, 37], applications considered in this paper consist of three types of participants: ① *data provider*, ② *computing server*, and ③ *data consumer*. In particular, data providers periodically send encrypted/shared messages to the servers for certain secure computation tasks. Then, two non-colluded computing servers conduct secure computation over encrypted/shared messages from data providers. The final computation is typically shared between servers at the end of secure computation. Finally, data consumers receive the computation result from the servers. In many cases, consumers are the computing servers themselves. We also use *client* to denote the data provider for short.

Threat model. We assume two servers are non-colluded. Servers and data contributors are semi-honest, meaning that they will faithfully follow protocol specification but may try to learn more information from protocol execution.

3 Preliminary

3.1 Notations

We use λ and κ to denote computational and statistical security parameters, respectively. $n = \text{poly}(\lambda)$ denotes that $n \in \mathbb{N}$ is polynomial in λ . $\text{negl}(\cdot)$ denotes some unspecified negligible function. Let $a \leftarrow b$ denote that a is assigned with value b . For a set A , $a \xleftarrow{\$} A$ denotes that a is uniformly randomly sampled from A . We use bold lowercase letters $\mathbf{a}$ to denote a vector. $\mathbb{G}$ denotes an Abelian group and $\mathbb{F}$ denotes a finite field. We use $\llbracket x \rrbracket$ to denote an additive linear secret sharing (LSS) of $x \in \mathbb{G}$. In the two-party case, $\llbracket x \rrbracket$ denotes the party P_b holds a share $\llbracket x \rrbracket_b$ such that $x = \llbracket x \rrbracket_0 + \llbracket x \rrbracket_1$. LSS supports affine transformation $\llbracket z \rrbracket \leftarrow \alpha \cdot \llbracket x \rrbracket + \beta \cdot \llbracket y \rrbracket + \gamma$ using public $\alpha, \beta, \gamma \in \mathbb{G}$ via *local* computation. We define *secret-shared multiplication* $\llbracket z \rrbracket \leftarrow \llbracket x \rrbracket \cdot \llbracket y \rrbracket$, which requires interaction between the parties; $\mathbb{G}$ is a ring or field to support multiplication.

Function family. A function family $\mathcal{F}$ contains a set of functions of the same type. For a function $f \in \mathcal{F}$, we use $\hat{f}$ to denote the *description* of f . $\hat{f}$ contains $\mathbb{G}_{\text{in}}$, $\mathbb{G}_{\text{out}}$, and a size parameter $\mathbb{S}$. Below, we show several commonly used function families in FSS.

Single-point function. A single-point function $f_{\alpha, \beta}(x) : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ outputs β for $x = \alpha$ and 0 otherwise.

Multi-point function. A t -point function $f_{A, B}(x) : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ for $A = \{\alpha_1, \dots, \alpha_t\} \in \mathbb{G}_{\text{in}}^t$ and $B = \{\beta_1, \dots, \beta_t\} \in \mathbb{G}_{\text{out}}^t$ evaluates to β_i on input α_i for $i \in [1, t]$ and 0 otherwise.

Comparison function. A comparison function $f_{\alpha, \beta}^<(x) : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ evaluates to β on input $x < \alpha$ and 0 otherwise.

Predicate function. A predicate function $f(x) : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ outputs 0/1 $\in \mathbb{G}_{\text{out}}$ for any $x \in \mathbb{G}_{\text{in}}$. Note that single-point

functions, multi-point functions, and comparison functions with only binary outputs are all predicate functions.

Definition 2 (Tuple product of functions). *Let $f : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ and $p : \mathbb{G}_{\text{in}} \rightarrow \mathbb{R}$ be two functions defined over the same domain $\mathbb{G}_{\text{in}}$. The tuple product function $g = f \times p : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}} \times \mathbb{R}$ is defined as $g(x) = (f(x), p(x))$, $\forall x \in \mathbb{G}_{\text{in}}$.*

3.2 Function Secret Sharing

Definition 3 (Function Secret Sharing [17]). *A two-party FSS scheme for function family $\mathcal{F}$ is defined as follows.*

- $(k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f})$. On input 1^λ and description $\hat{f} \in \{0, 1\}^*$ of a function $f : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}} \in \mathcal{F}$, output a key pair (k_0, k_1) .
- $\llbracket f(x) \rrbracket_b \leftarrow \text{Eval}(b, k_b, x)$. On input a party identifier $b \in \{0, 1\}$, key k_b , and $x \in \mathbb{G}_{\text{in}}$, output a share $\llbracket f(x) \rrbracket_b \in \mathbb{G}_{\text{out}}$.

An FSS scheme $(\text{Gen}, \text{Eval})$ is secure for the function family $\mathcal{F}$ with leakage $\text{Leak} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ if the following holds:

- **Correctness.** For any function $f \in \mathcal{F}$ and any $x \in \mathbb{G}_{\text{in}}$,

$$\Pr \left[\begin{array}{l} (k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f}) : \\ \text{Eval}_0(0, k_0, x) + \text{Eval}_1(1, k_1, x) = f(x) \end{array} \right] = 1$$

- **Security.** For any PPT adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that for any $f \in \mathcal{F}$ and $b \in \{0, 1\}$:

$$\left| \Pr \left[(k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f}) : \mathcal{A}(1^\lambda, k_b) = 1 \right] - \Pr \left[k_b \leftarrow \mathcal{S}(1^\lambda, b, \text{Leak}(\hat{f})) : \mathcal{A}(1^\lambda, k_b) = 1 \right] \right| \leq \text{negl}(\lambda),$$

where $\text{Leak}(\hat{f})$ reveals domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$ of f .

FSS with pseudorandom share. We say an FSS scheme for $\mathcal{F}$ has pseudorandom-share property if for any $f \in \mathcal{F}$, $b \in \{0, 1\}$, and k_b with $(k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \hat{f})$ over the coin of Gen , and any $x \in \mathbb{G}_{\text{in}}$, the distribution of $\llbracket f(x) \rrbracket_b = \text{Eval}(b, k_b, x)$ is computationally indistinguishable from uniform distribution over $\mathbb{G}_{\text{out}}$, which requires $\text{Eval}(b, k_b, \cdot)$ behaves like a pseudorandom function. This property is formalized by Boyle *et al.* [17] for poly-spanning function families (§4.1, [17]), which includes useful and practical functions considered in existing FSS work, such as point functions, comparison functions, etc.

3.3 Key-Homomorphic PRF

Definition 4 (B -almost Key-Homomorphic Pseudorandom Function (KH-PRF)). *Let $F : \mathbb{K} \times \mathbb{X} \rightarrow \mathbb{Y}$ be a secure pseudorandom function and $(\mathbb{K}, +_{\mathbb{K}})$ is a group. We say f is B -almost key-homomorphic if for any $k_1, k_2 \in \mathbb{K}$ and $x \in \mathbb{X}$, there exists a bounded error $e_{\text{kh}} \in [0, B]$ for some $B \in \mathbb{G}_{\text{out}}$ such that*

$$F(k_1 +_{\mathbb{K}} k_2, x) = F(k_1, x) +_{\mathbb{Y}} F(k_2, x) +_{\mathbb{Y}} e_{\text{kh}}.$$

Special zero-key property. We say that a KH-PRF F has the *special zero-key* property if $F(0, x) = 0 \in \mathbb{G}_{\text{out}}$ holds for any $x \in \mathbb{G}_{\text{in}}$. We note that the *special zero-key* property does not generally hold for all KH-PRFs or any group $\mathbb{G}_{\text{out}}$. For example, the DDH-based PRF [50] $F(k, x) = H(x)^k$, assuming a random oracle $H : \{0, 1\}^* \rightarrow \mathbb{G}$, is a KH-PRF, but $F(0, x) = H(x)^0 = 1_{\mathbb{G}}$.

LWR-based KH-PRF. We can instantiate KH-PRF with a special zero-key from the seminal LWR-based KH-PRF [5]. Formally, let $q \geq p \geq 2$ be two integers. The rounding function $\lfloor x \rfloor_p : \mathbb{Z}_q \rightarrow \mathbb{Z}_p$ outputs $\lfloor x \rfloor_p = i$, where i is the largest multiple of q/p that does not exceed x . Let $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^d$ be a hash function modeled as a random oracle. For $\mathbf{k} \in \mathbb{Z}_q^d$, $F_{\text{LWR}}(\mathbf{k}, x) : \mathbb{Z}_q^d \times \{0, 1\}^* \rightarrow \mathbb{Z}_p$ is defined as:

$$F_{\text{LWR}}(\mathbf{k}, x) = \lfloor \langle H_1(x), \mathbf{k} \rangle \rfloor_p.$$

Note that $F_{\text{LWR}}(\mathbf{k}, x)$ is B -almost key-homomorphic with $B = 1$, as the rounding introduces 1 bit of error at most. We note that LWR-based KH-PRF has a special zero-key.

4 Streaming Function Secret Sharing

This section details our SFSS constructions.

4.1 Possible Approaches

Recall that FSS provides a mechanism to additively share $f(x)$, for a secret-shared function f and any public $x \in \mathbb{G}_{\text{in}}$, via *non-interactive* evaluation. SFSS extends this capability by additively sharing $f(x) \cdot m$ *non-interactively*, where $x \in \mathbb{G}_{\text{in}}$ is public and c is a streaming ciphertext encrypting m . Although no prior work formalizes this streaming variant of FSS, several approaches resemble some aspects of this notion. However, these methods do not align with the non-interactive semantics of (S)FSS and therefore fail to realize SFSS.

Homomorphic-encryption-based approach. Linear homomorphic encryption (LHE) seems to be a match to compute $f(x) \cdot m$. We compare the conceptual differences between SFSS and LHE-based approaches.

Suppose we have an LHE scheme with message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and $c = \text{Enc}(\text{pk}, m)$ be the ciphertext of message m . LHE supports homomorphic addition ($\boxplus$) and scalar multiplication ($\boxtimes$) over ciphertexts:

- $\text{Enc}(\text{pk}, x) \boxplus \text{Enc}(\text{pk}, y) = \text{Enc}(\text{pk}, x +_{\mathcal{M}} y)$
- For any $y \in \mathcal{M}$, $\text{Enc}(\text{pk}, x) \boxtimes y = \text{Enc}(\text{pk}, x \cdot_{\mathcal{M}} y)$

A simple “FSS+LHE” solution is sketched as follows:

- A streaming message m is encrypted as $c = \text{Enc}(\text{pk}, m)$.
- For $b \in \{0, 1\}$, evaluate FSS to obtain $\llbracket f(x) \rrbracket_b$ over $\mathcal{M}$. Compute $c'_b \leftarrow c \boxtimes \llbracket f(x) \rrbracket_b$. By definition, $c'_0 \boxplus c'_1 = \text{Enc}(\text{pk}, f(x) \cdot_{\mathcal{M}} m)$.

As we can see, ciphertexts c'_0 and c'_1 are $\boxplus$ -shared over the ciphertext space $\mathcal{C}$. While it is possible to recover or additively share $f(x) \cdot m$ via secret-shared decryption protocols for certain LHE schemes, this approach is inherently interactive. We provide a concrete example in Appendix §A.3.

Vizard’s approach. Vizard [24] resorts to a similar “FSS+LHE” concept but combines DPF over an (symmetric) homomorphic stream encryption (HSE) scheme [23]. However, we find that Vizard cannot achieve its claimed security and efficiency goals, let alone SFSS.

In Vizard, a client i generates a pair of DPF keys for a function $f^{(i)} \in \mathcal{F}$ and distributes the keys between two servers, serving as a secret policy function. For the j -th message, the client i shares the streaming message $\llbracket m_j^{(i)} \rrbracket$ between the servers. To share $\llbracket f^{(i)}(x) \cdot m_j^{(i)} \rrbracket$ for any $x \in \mathbb{G}_{\text{in}}$, the servers non-interactively generate a random sharing $\llbracket r_j^{(i)} \rrbracket$, compute $\llbracket c_j^{(i)} \rrbracket = \llbracket m_j^{(i)} \rrbracket + \llbracket r_j^{(i)} \rrbracket$, and open $c_j^{(i)} = m_j^{(i)} + r_j^{(i)}$, where $r_j^{(i)}$ perfectly hides $m_j^{(i)}$. Then, the servers jointly compute

$$\llbracket f^{(i)}(x) \cdot m_j^{(i)} \rrbracket \leftarrow c_j^{(i)} \cdot \llbracket f^{(i)}(x) \rrbracket - \llbracket f^{(i)}(x) \rrbracket \cdot \llbracket r_j^{(i)} \rrbracket.$$

However, computing $\llbracket f^{(i)}(x) \rrbracket \cdot \llbracket r_j^{(i)} \rrbracket$ requires *interactive* secret-shared multiplication.

Directly applying the above method for policy-hiding aggregation over ciphertexts from N_c clients incurs $O(N_c)$ communication complexity, as the servers must perform secret-shared multiplication per data stream. To reduce the overhead, Vizard *reuses* a mask r_j for all N_c clients in round j , i.e., $r_j^{(i)} = r_j$ for all $i \in [N_c]$, thus we have $c_j^{(i)} = m_j^{(i)} + r_j$ for all $i \in [N_c]$. In this way, the server computes the aggregation

$$\llbracket s \rrbracket \leftarrow \sum_{i \in [N_c]} c_j^{(i)} \cdot \llbracket f^{(i)}(x) \rrbracket - \left(\sum_{i \in [N_c]} \llbracket f^{(i)}(x) \rrbracket \right) \cdot \llbracket r_j \rrbracket.$$

Now the servers only conduct one single secret-shared multiplication ($\sum_{i \in [N_c]} \llbracket f^{(i)}(x) \rrbracket \cdot \llbracket r_j \rrbracket$) with $O(1)$ communication complexity.

Unfortunately, this optimization introduces a security flaw, as the secret key r_j is reused to encrypt messages for all clients. Thus, for any $i_1, i_2 \in [N_c]$, we have

$$m_j^{(i_1)} - m_j^{(i_2)} = c_j^{(i_1)} - c_j^{(i_2)},$$

which breaks security trivially. It is unclear how to fix the flaw without blowing up to $O(N_c)$ communication complexity.

Summary. Existing approaches are interactive, not aligned with SFSS non-interaction semantics. In general, communication and interaction are more likely to be efficiency bottlenecks for real-world applications. Though the LHE-based approach may be promising for specific aggregation applications, SFSS evaluation generates additive sharing without interaction, which would benefit broader applications beyond

aggregation. Vizard necessitates communication to generate a ciphertext, and its evaluation phase is interactive (and flawed). Looking ahead, we design non-interactive streaming encryption and non-interactive evaluation as required by SFSS.

4.2 SFSS for Single-point Functions

We first propose an SFSS compiler for single-point functions $f_{\alpha, \beta}$, called streaming DPF (SDPF). We start with the case $\beta = 1$ and extend it to the non-binary case in Appendix §A.1.

4.2.1 Streaming DPF with $\beta = 1$

Technique overview. Fig. 2 sketches the high-level idea, and Fig. 3 presents a formal scheme. At the core are *non-interactive* PRF-based streaming masking and unmasking mechanisms, which use the pseudorandom shares from DPF as the PRF keys. Now we elaborate our idea step by step.

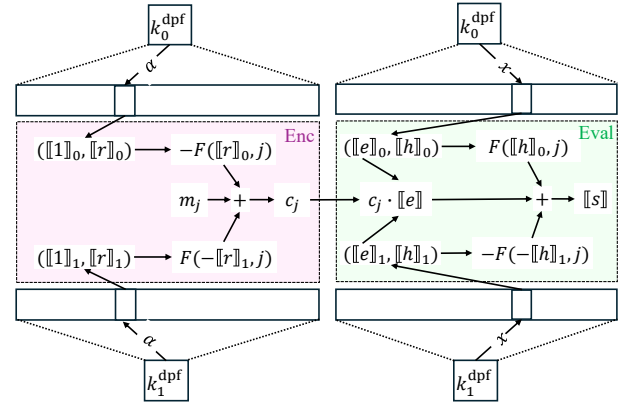


Figure 2: **Sketched streaming encryption and evaluation.** Two pseudorandom shares $(\llbracket r \rrbracket_0, \llbracket r \rrbracket_1)$ evaluated at α serve as the PRF keys over counter j to derive two masks for encryption. For evaluation over any $x \in \mathbb{G}_{\text{in}}$, a local unmasking strategy using the pseudorandom shares $(\llbracket e \rrbracket_b, \llbracket h \rrbracket_b)$ essentially shares $\llbracket s \rrbracket$ such that $s = m_j$ if $x = \alpha$, and $s = 0$ otherwise.

First, we rely on a pseudorandom distributed point function (PDPF) to design SDPF. PDPF additionally embeds a pseudorandom payload r at the target point α :

$$f_{\alpha, (1, r)}(x) = \begin{cases} (1, r), & x = \alpha \\ (0, 0), & x \neq \alpha \end{cases}$$

We note that it is essentially free to obtain PDPF from existing PRG-based DPF constructions [11, 17, 18, 39]. And we use the DPF from [18] in our implementation. The remaining question is how to compile PDPF to SDPF.

SDPF setup phase simply generates a pair of PDPF keys $(k_0^{\text{dpf}}, k_1^{\text{dpf}})$ via DPF key generation. From the DPF keys, as depicted in Fig. 2, we use the DPF pseudorandom shares

Parameters: PRF $F : \{0, 1\}^\lambda \times \mathbb{G}_{\text{ctr}} \rightarrow \mathbb{G}_{\text{out}}$ with $|\mathbb{G}_{\text{ctr}}| \geq 2^\lambda$; Function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$ of single-point functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$; Function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda}$ of single-point functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda$; DPF scheme Π_{DPF} for function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda}$.

$\text{Gen}(1^\lambda, \hat{f}_{\alpha, 1} \in \mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}})$:

- 1: Set $\text{ctr} = 1, \text{st}_f \leftarrow \{\text{ctr}\}$
- 2: $r \xleftarrow{\$} \{0, 1\}^\lambda, (k_0^{\text{dpf}}, k_1^{\text{dpf}}) \leftarrow \Pi_{\text{DPF}}.\text{Gen}(1^\lambda, \hat{f}_{\alpha, (1, r)})$
- 3: For $b \in \{0, 1\}$, $(\llbracket 1 \rrbracket_b, \llbracket r \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{\text{dpf}}, \alpha)$
- 4: $k_e = \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}, k_0 \leftarrow \{k_0^{\text{dpf}}\}, k_1 \leftarrow \{k_1^{\text{dpf}}\}$
- 5: Return $(\text{st}_f, k_e, k_0, k_1)$.

$\text{Enc}(\text{st}_f, k_e, m)$:

- 1: Parse $\text{st}_f = \{\text{ctr}\}, k_e = \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}$.
- 2: $c_{\text{ctr}} \leftarrow m - F(\llbracket r \rrbracket_0, \text{ctr}) + F(-\llbracket r \rrbracket_1, \text{ctr})$
- 3: Set $c \leftarrow (\text{ctr}, c_{\text{ctr}})$, update $\text{st}_f \leftarrow \{\text{ctr} + 1\}$
- 4: Return (st_f, c)

$\text{Eval}(b, k_b, x, c)$:

- 1: Parse $c = (j, c_j)$ and $k_b = \{k_b^{\text{dpf}}\}$
- 2: $(\llbracket e \rrbracket_b, \llbracket h \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{\text{dpf}}, x)$
- 3: $\llbracket s \rrbracket_b \leftarrow \llbracket e \rrbracket_b \cdot c_j + (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, j)$
- 4: Return $(\llbracket s \rrbracket_b)$

Figure 3: A SDPF construction with $\beta = 1$.

evaluated at α as a pair of PRF keys to derive pseudorandom masks for streaming encryption. Specifically, let

$$(\llbracket 1 \rrbracket_b, \llbracket r \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b, \alpha)$$

for $b \in \{0, 1\}$, the pseudorandom shares $(\llbracket r \rrbracket_0, \llbracket r \rrbracket_1)$ are used as the PRF keys. To encrypt the j -th streaming message m_j , we take the following masking strategy:

$$c_j \leftarrow m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j).$$

Namely, m_j is double-masked by two pseudorandom masks derived from $\llbracket r \rrbracket_0$ and $\llbracket r \rrbracket_1$ over the counter j using PRF F .

It remains to design the SFSS evaluation to correctly share $f_{\alpha, 1}(x) \cdot m_j$, by evaluation using k_b and c_j for any $x \in \mathbb{G}_{\text{in}}$. As depicted in Fig. 2, we use the following unmasking strategy:

$$(\llbracket e \rrbracket_b, \llbracket h \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b, x),$$

$$\llbracket s \rrbracket_b \leftarrow \llbracket e \rrbracket_b \cdot c_j + (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, j).$$

We prove $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = f_{\alpha, 1}(x) \cdot m_j$ for any $x \in \mathbb{G}_{\text{in}}$ as follows.

Correctness. We prove $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = m_j$ for $x = \alpha$ and $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = 0$ for $x \neq \alpha$. To see this, $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = \llbracket e \rrbracket_0 \cdot c_j + (-1)^0 \cdot F((-1)^0 \cdot \llbracket h \rrbracket_0, j) + \llbracket e \rrbracket_1 \cdot c_j + (-1)^1 \cdot F((-1)^1 \cdot \llbracket h \rrbracket_1, j) = e \cdot (m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j)) + F(\llbracket h \rrbracket_0, j) - F(-\llbracket h \rrbracket_1, j)$. There are two cases depending on whether $x = \alpha$ or $x \neq \alpha$:

- Case $x = \alpha$: In this case, we have $e = 1$, $\llbracket h \rrbracket_0 = \llbracket r \rrbracket_0$, and $\llbracket h \rrbracket_1 = \llbracket r \rrbracket_1$, thus $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j) + F(\llbracket h \rrbracket_0, j) - F(-\llbracket h \rrbracket_1, j) = m_j$.
- Case $x \neq \alpha$: In this case, we have $e = 0$, $\llbracket h \rrbracket_0 + \llbracket h \rrbracket_1 = 0 \Leftrightarrow \llbracket h \rrbracket_0 = -\llbracket h \rrbracket_1$, thus $e \cdot c_j + F(\llbracket h \rrbracket_0, j) - F(-\llbracket h \rrbracket_1, j) = 0$.

Overall, $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = f_{\alpha, 1}(x) \cdot m_j, \forall x \in \mathbb{G}_{\text{in}}$.

Security. Intuitively, a secure DPF scheme ensures that k_b is pseudorandom and hides α and r . $\llbracket r \rrbracket_{1-b}$ is a pseudorandom share. PRF F guarantees that $F(\llbracket r \rrbracket_{1-b}, j)$ is pseudorandom. As a result, the distribution of the ciphertext $c_j = m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j)$ is computationally indistinguishable from uniform distribution over $\mathbb{G}_{\text{out}}$, as $F(\llbracket r \rrbracket_{1-i}, j)$ serves as a pseudorandom mask that hides other values. Therefore, the tuple $(k_b, c_1, c_2, \dots)$ is computationally indistinguishable from random in the key and ciphertext spaces. Note that ctr should never repeat. Thus, we take sufficiently large $\mathbb{G}_{\text{ctr}}$ and re-setup SFSS keys if necessary; the same goes for other SFSS constructions. Theorem 1 formalizes the security intuition, with a full proof in Appendix §D.1.

Theorem 1. Let Π_{DPF} be a DPF scheme with pseudorandom-share property for a single-point function family with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda$, PRF $F : \{0, 1\}^\lambda \times \mathbb{G}_{\text{ctr}} \rightarrow \mathbb{G}_{\text{out}}$ with sufficiently large $\mathbb{G}_{\text{ctr}}$, then Π_{SDPF} in Fig. 3 is an adaptively secure SDPF scheme with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$.

4.3 SFSS for Predicate Functions

SDPF introduces little overhead compared to DPF, as it only additionally relies on PRFs. However, this framework cannot easily be extended beyond single-point functions.

4.3.1 Challenge

To elaborate the challenge concretely, we show an attempt of designing SFSS for a multi-point function $f_{A, B}$ with $A = \{\alpha_1, \alpha_2\}$ and $B = \{\beta_1, \beta_2\}$, where $\alpha_1 \neq \alpha_2$ and $\beta_1 = \beta_2 = 1$. As before, we define an augmented two-point function $f_{A, (B, r)} : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda$ as

$$f_{A, (B, r)}(x) = \begin{cases} (1, r), & x \in A \\ (0, 0), & x \notin A, \end{cases}$$

and generate the MDPF keys $(k_0, k_1) \leftarrow \Pi_{\text{MDPF}}.\text{Gen}(1^\lambda, \hat{f})$ via any existing MDPF scheme such as [16].

The remaining question is how to perform streaming encryption. In particular, how to choose the binding randomness used in the encryption process, and more importantly, how to correctly unmask it during evaluation. Suppose the correlated randomness associated with index α_1 is selected as $(\llbracket \beta_1 \rrbracket_b, \llbracket r \rrbracket_b) \leftarrow \Pi_{\text{MDPF}}.\text{Eval}(b, k_b, \alpha_1)$, and the encryption algorithm encrypts the j -th streaming message as

$c_j \leftarrow m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j)$ as before. Similarly, the evaluation algorithm $\text{Eval}(b, k_b, (j, c_j), x)$ computes:

$$\begin{aligned} (\llbracket e \rrbracket_b, \llbracket h \rrbracket_b) &\leftarrow \Pi_{\text{MDPF}}.\text{Eval}(b, k_b, x), \\ \llbracket s \rrbracket_b &\leftarrow \llbracket e \rrbracket_b \cdot c_j + (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, j). \end{aligned}$$

One can verify that correctness still holds for $x \neq \alpha_2$. However, correctness fails to hold at $x = \alpha_2$. To see this, $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = e \cdot c_j + F(\llbracket h \rrbracket_b, j) - F(-\llbracket h \rrbracket_b, j) = e \cdot (m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j)) + (F(\llbracket h \rrbracket_0, j) - F(-\llbracket h \rrbracket_1, j)) = m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j) + F(\llbracket h \rrbracket_0, j) - F(-\llbracket h \rrbracket_1, j)$. Since the shares evaluated at α_1 and α_2 are pseudorandom and inherently randomized, the shares $\llbracket r \rrbracket_0 \neq \llbracket h \rrbracket_0$ and $\llbracket r \rrbracket_1 \neq \llbracket h \rrbracket_1$ hold with overwhelming probability – no way to take off the masks as we do for SDPF.

4.3.2 New KH-PRF-based Solution

The prior approach attempted to embed correlated randomness at non-zero outputs to derive streaming masks, but failed to ensure correctness. *The issue arises as the embedded randomness r is not directly used to derive masks; instead, its randomized shares are used to derive masking keys, leading to inconsistency during evaluation.* Hence, the evaluation cannot correctly take off the masks from the ciphertext. An intuition comes: can we derive random masks from r directly?

We propose a new masking strategy that directly derives masks from a shared secret key, allowing unmasking to be performed in a secret-shared manner. We achieve this idea via *key-homomorphic pseudorandom function* (KH-PRF), which supports local PRF evaluation over shared keys. By embedding a KH-PRF key in the function output, servers can recover consistent key shares at evaluation points and use key homomorphism to jointly derive a shared PRF output, enabling secure and correct masking and unmasking.

The proposed construction. Fig. 4 describes the formal construction. To encode the KH-PRF key into a predicate function, we first define a payload function $p : \mathbb{G}_{\text{in}} \rightarrow \mathbb{K}_{\text{kh}}$.

$$p(x) = \begin{cases} k_{\text{kh}}, & f(x) = 1, \\ 0, & f(x) = 0, \end{cases}$$

where k_{kh} is the KH-PRF key. Then, we define an argued function $h = f \times p$ as the tuple product of f and p (c.f., Def. 2). By definition, we have

$$h(x) = \begin{cases} (1, k_{\text{kh}}), & f(x) = 1, \\ (0, 0), & f(x) = 0. \end{cases}$$

Let Π_{FSS} be an FSS scheme for the function family $\mathcal{F}$ that h belongs to. We can generate a pair of FSS keys $(k_0^{\text{FSS}}, k_1^{\text{FSS}}) \leftarrow \Pi_{\text{FSS}}.\text{Gen}(1^\lambda, \hat{h})$. To encrypt the j -th streaming message m_j , the streaming encryption algorithm simply computes:

$$c_j \leftarrow m_j + F(k_{\text{kh}}, j),$$

where $F(k_{\text{kh}}, j)$ computationally hides m_j .

The SFSS evaluation should share $f(x) \cdot m_j$ by correctly taking off the mask $F(k_{\text{kh}}, j)$ in a secret-shared fashion. To this end, the SFSS evaluation algorithm computes

$$\begin{aligned} (\llbracket e \rrbracket_b, \llbracket k \rrbracket_b) &\leftarrow \Pi_{\text{FSS}}.\text{Eval}(b, k_b^{\text{fss}}, x), \\ \llbracket s \rrbracket_b &\leftarrow \llbracket e \rrbracket_b \cdot c_j - F(\llbracket k \rrbracket_b, j). \end{aligned}$$

We now argue its correctness and security as follows.

Correctness. For any $x \in \mathbb{G}_{\text{in}}$, we show that $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = f(x) \cdot m + e_{\text{kh}}$, where $e_{\text{kh}} \in [0, B]$ is a small and bounded error. Thus, our KH-PRF-based SFSS satisfies the B -bounded correctness, and we use KH-PRF with a special zero-key property (c.f. §3.3). By the definition, we have

$$\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = e \cdot c_j - F(\llbracket k \rrbracket_0, j) - F(\llbracket k \rrbracket_1, j).$$

There are two cases depending on whether $f(x) = 0$ or 1:

- Case $f(x) = 1$: We have $e = 1$ and $k = k_{\text{kh}}$, hence $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = m_j + F(k_{\text{kh}}, j) - (F(\llbracket k_{\text{kh}} \rrbracket_0, j) + F(\llbracket k_{\text{kh}} \rrbracket_1, j)) = m_j + F(k_{\text{kh}}, j) - (F(k_{\text{kh}}, j) - e_{\text{kh}}) = m_j + e_{\text{kh}}$,
- Case $f(x) = 0$: We have $e = 0$, $k = 0$, and $\llbracket s \rrbracket_0 + \llbracket s \rrbracket_1 = 0 - (F(\llbracket 0 \rrbracket_0, j) + F(\llbracket 0 \rrbracket_1, j)) = -(F(0, j) - e_{\text{kh}}) = e_{\text{kh}}$,

Parameters: A B -almost KH-PRF $F : \mathbb{K}_{\text{kh}} \times \mathbb{G}_{\text{ctr}} \rightarrow \mathbb{G}_{\text{out}}$; A function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$ for predicate functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$; A function family $\mathcal{P}_{\mathbb{G}_{\text{in}}, \mathbb{K}_{\text{kh}}}$ for payload functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{K}_{\text{kh}}$; A function family $\mathcal{H} = \{f \times p : f \in \mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}, p \in \mathcal{P}_{\mathbb{G}_{\text{in}}, \mathbb{K}_{\text{kh}}}\}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}} \times \mathbb{K}_{\text{kh}}$; An FSS scheme Π_{FSS} for the function family $\mathcal{H}$; $|\mathbb{G}_{\text{ctr}}| \geq 2^\lambda$.

Gen($1^\lambda, \hat{f} \in \mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$):

- 1: $k_{\text{kh}} \xleftarrow{\$} \mathbb{K}_{\text{kh}}, \text{ctr} = 1$.
- 2: Define function $p \in \mathcal{P}$ s.t. $p(x) = k_{\text{kh}} \cdot f(x), \forall x \in \mathbb{G}_{\text{in}}$.
- 3: Let $h = f \times p$, compute $(k_0^{\text{fss}}, k_1^{\text{fss}}) \leftarrow \Pi_{\text{FSS}}.\text{Gen}(1^\lambda, \hat{h})$.
- 4: $\text{st}_f \leftarrow \{\text{ctr}\}, k_e \leftarrow \{k_{\text{kh}}\}, k_0 \leftarrow \{k_0^{\text{fss}}\}, k_1 \leftarrow \{k_1^{\text{fss}}\}$.
- 5: Return $(\text{st}_f, k_e, k_0, k_1)$.

Enc(st_f, k_e, m):

- 1: Parse $\text{st}_f = \{\text{ctr}\}$ and $k_e = \{k_{\text{kh}}\}$.
- 2: $c_{\text{ctr}} \leftarrow m + F(k_{\text{kh}}, \text{ctr})$.
- 3: Set $c \leftarrow (\text{ctr}, c_{\text{ctr}})$, update $\text{st}'_f = \{\text{ctr} + 1\}$.
- 4: Return (st'_f, c) .

Eval(b, k_b, x, c):

- 1: Parse $c = (j, c_j)$ and $k_b = \{k_b^{\text{fss}}\}$.
- 2: $(\llbracket e \rrbracket_b, \llbracket k \rrbracket_b) \leftarrow \Pi_{\text{FSS}}.\text{Eval}(b, k_b^{\text{fss}}, x)$.
- 3: $\llbracket s \rrbracket_b \leftarrow \llbracket e \rrbracket_b \cdot c_j - F(\llbracket k \rrbracket_b, j)$.
- 4: Return $(\llbracket s \rrbracket_b)$.

Figure 4: A SFSS framework for predicate functions.

where $e_{\text{kh}} \in [0, B]$ is the bounded error introduced from B -almost KH-PRF. The last equation holds in Case 2 as $F(0, j) = 0$, following the required *special zero-key* property.

Security. We prove that any single SFSS key k_b and a polynomially many ciphertexts reveal no more information than allowed. Intuitively, the security of FSS for the predicate function family ensures that k_b^{fss} is pseudorandom, thus hiding the function within its family, which implies that the embedded KH-PRF key k_{kh} is computationally hidden. Then the security of KH-PRF F ensures that $F(k_{\text{kh}}, j)$ is pseudorandom. Consequently, the distribution of the ciphertext $c_j = m_j + F(k_{\text{kh}}, j)$ is computationally indistinguishable from uniform distribution over $\mathbb{G}_{\text{out}}$. Thus, the tuple $(k_b, c_1, c_2, \dots, c_q)$ is computationally indistinguishable from a uniform distribution over the key and ciphertext spaces. Theorem 2 formalizes security, with a proof in Appendix §D.2.

Theorem 2. *Given an FSS scheme Π_{FSS} for a predicate function family $\mathcal{F}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$, a B -almost KH-PRF $F : \mathbb{K}_{\text{kh}} \times \mathbb{G}_{\text{ctr}} \rightarrow \mathbb{G}_{\text{out}}$ with the special zero-key property, then the construction in Fig. 4 is an adaptively secure SFSS scheme of B -bounded correctness with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$.*

Error reduction & Removal. The shared output from the KH-PRF-based SFSS contains a bounded error. Looking ahead, errors will accumulate proportionally with the number of data streams when SFSS is used for aggregation. While the errors are acceptable for applications where approximate results are sufficient, we propose a “scale-then-truncate” method inspired by [46], reducing the error to at most one bit. We can further use a simple two-party protocol to completely remove the one-bit error. Refer to Appendix §A.2 for details.

4.3.3 A Concrete Construction: From MDPF to SMDPF

To understand the compiler from Fig. 4 concretely, we show how to compile MDPF to streaming MDPF (SMDPF).

Step 1: Decompose a t -point function to t single-point functions. For any t -point function $f_{A,B}(x) : \mathbb{G}_{\text{in}} \rightarrow \mathbb{G}_{\text{out}}$ with $A = \{\alpha_1, \dots, \alpha_t\} \in \mathbb{G}_{\text{in}}'$ and $B = \{1, \dots, 1\} \in \mathbb{G}_{\text{out}}'$, we can always decompose $f_{A,B}$ as t single-point functions: $f_{A,B} = \sum_{i \in [t]} f_{\alpha_i, 1}^{(i)}$, where $f_{\alpha_i, 1}^{(i)}(x)$ outputs 1 only at $x = \alpha_i$ and 0 otherwise.

Step 2: Couple each single-point function with a KH-PRF secret key. Following our KH-PRF-based compiler, we extend the range of the i -th single-point function and define a new argued single-point function:

$$f_{\alpha_i, (1, k_{\text{kh}})}^{(i)}(x) = \begin{cases} (1, k_{\text{kh}}), & x = \alpha_i, \\ (0, 0), & x \neq \alpha_i. \end{cases}$$

Let $h = \sum_{i \in [t]} f_{\alpha_i, (1, k_{\text{kh}})}^{(i)}$, it is not hard to see:

$$h(x) = \sum_{i \in [t]} f_{\alpha_i, (1, k_{\text{kh}})}^{(i)}(x) = \begin{cases} (1, k_{\text{kh}}), & x \in A, \\ (0, 0), & x \notin A. \end{cases}$$

Step 3: Define a DPF for each argued single-point function. The remaining is simply designing a DPF for each argued single-point function $f_{\alpha_i, (1, k_{\text{kh}})}^{(i)}$. These t DPFs sum together to form an FSS for the argued t -point function h . Then, following the compiler from Fig. 4, we directly obtain a KH-PRF-based SFSS for MDPF.

The above strategy can be applied to compile recent MDPF constructions [16] with improved efficiency.

Cost analysis. We analyze the overhead from SMDPF compared with MDPF. Using the DPF construction from [18], encoding the KH-PRF key k_{kh} requires one more correction word of size $|\mathbb{K}_{\text{kh}}|$ for each DPF key. Let $\text{Keysize}(\Pi)$ be the keysize of a scheme Π , then we have $\text{Keysize}(\Pi_{\text{SMDPF}}) = \text{Keysize}(\Pi_{\text{MDPF}}) + t \cdot |\mathbb{K}_{\text{kh}}|$. As for evaluation, SMDPF additionally conducts the KH-PRF evaluation that requires inner-product computation between vectors in the LWR-based KH-PRF. The KH-PRF key contributes to the main overhead in key size, and the KH-PRF evaluation contributes additional computational overhead. Nevertheless, KH-PRF-based SFSS, including SMDPF, is still promising for certain applications as we will show in §5.1.2 and §6.

4.4 SFSS Extensions

SFSS for generic functions. Based on SFSS for predicate functions, we further propose SFSS for generic functions. The high-level idea is to decompose each function output as a binary vector and then apply our SFSS technique for predicate functions. We move the details to Appendix §A.4.

Stateless SFSS. All prior SFSS constructions are state-based as they require the client to maintain a non-repeatable counter. We can slightly modify the construction with stateless encryption by randomly sampling a fresh nonce from $\mathbb{G}_{\text{ctr}}$ each time. This requires $\mathbb{G}_{\text{ctr}}$ to be large enough to ensure the collusion probability $q^2/|\mathbb{G}_{\text{ctr}}|$ (from the birthday bound) is negligible.

5 SFSS-based Applications

5.1 Policy-hiding Aggregation

Policy-hiding aggregation [24] allows clients to attach policy functions to their data streams, restricting the condition under which the streaming message can be used for aggregation. It aims to protect both the data stream and client-defined policy (*i.e.*, metadata privacy). Fig. 5 defines the ideal functionality with commands SETUP, SEND, and AGGREGATE. In SETUP, a client registers its policy function. In SEND, client i submits its j -th streaming message $m_j^{(i)}$. AGGREGATE computes the result as $\sum_{i \in [N_c]} f^{(i)}(\text{att}) \cdot m_j^{(i)}$, where $f^{(i)}$ is the client’s policy function and att denotes the attribute for an aggregation task. The functionality does not reveal stream messages, policy, or whether a stream is involved in an aggregation task.

Functionality $\mathcal{F}_{\text{phAgg}}$

Parameters: N_c denotes the number of clients; A policy function family $\mathcal{F}$; An attribute list $\mathcal{A}$; A dictionary Val storing policies; A dictionary Msg storing messages.

Functionality:

- **SETUP:** on input $(\text{setup}, i, f^{(i)} \in \mathcal{F})$ from client i , store $\text{Val}[i] \leftarrow f^{(i)}$.
- **SEND:** on input streaming message $(\text{send}, i, j, m_j^{(i)})$ from client i in round j , store $\text{Msg}[i, j] \leftarrow m_j^{(i)}$.
- **AGGREGATE:** on receiving an aggregation task $(\text{agg}, \text{att}, j)$ from an aggregation receiver, broadcast att to all parties, return $\sum_{i \in [N_c]} f^{(i)}(\text{att}) \cdot m_j^{(i)}$.

Figure 5: Ideal policy-hiding aggregation functionality

5.1.1 Existing Approaches

Two possible approaches realize policy-hiding aggregation.

Non-streaming FSS-based solution. We can apply non-streaming FSS as outlined in §2.1, by defining a weighted function with messages encoded as the policy function output, and sharing the function via a pair of FSS keys. However, the clients must send new FSS keys for each message, even though the policy remains unchanged. Moreover, servers must store all FSS keys for future aggregation. This results in high costs in communication and server-side storage.

Vizard’s solution. As discussed in §4.1, Vizard still requires inter-server communication during the message sending phase and the aggregation phase. To avoid linear communication in the aggregation phase, Vizard reuses the same key for masking messages from different users, which introduces a security flaw. Even the flawed Vizard requires $\omega(1)$ communication complexity in the aggregation phase; Vizard only achieves constant inter-server communication when *all* data streams contain the same number of messages.

5.1.2 SFSS-powered Policy-hiding Aggregation

SFSS provides a simple and modular design for policy-hiding aggregation, as shown in Fig. 6. When SFSS is configured by SDPF, our solution realizes the same functionality that Vizard failed to realize securely, meanwhile eliminating server-to-server communication in the sending and aggregation phases.

Table 1 compares the communication complexity of three different approaches. For single-point functions, we additionally compare with Vizard. For DPF-related phAgg, Vizard incurs linear inter-server communication during the sending phase and $\omega(1)$ communication during aggregation, while our construction eliminates inter-server interaction. Compared to the DPF-based baseline, our solution significantly reduces

Parameters: N_c denotes the number of clients; A function family $\mathcal{F}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$; A SFSS scheme Π_{SFSS} for $\mathcal{F}$.

Setup $(1^\lambda, \hat{f}^{(i)}; \perp; \perp)$:

- 1: The client generates SFSS keys:

$$(\text{st}_f^{(i)}, \text{ke}_e^{(i)}, \text{k}_0^{(i)}, \text{k}_1^{(i)}) \leftarrow \Pi_{\text{SFSS}}.\text{Gen}(1^\lambda, \hat{f}^{(i)})$$

- 2: Client i stores $\text{st}_f^{(i)}$ and $\text{ke}_e^{(i)}$ and sends $\text{k}_0^{(i)}$ and $\text{k}_1^{(i)}$ to the server 0 and 1, respectively. Server b stores $\text{k}_b^{(i)}$.

Send $((\text{st}_f^{(i)}, \text{ke}_e^{(i)}, m_j^{(i)}); \perp; \perp)$: ▷ j -th message

- 1: Client i runs

$$(\text{st}_f^{(i)}, c_j^{(i)}) \leftarrow \Pi_{\text{SFSS}}.\text{Enc}(\text{st}_f^{(i)}, \text{ke}_e^{(i)}, m_j^{(i)})$$

and sends $c_j^{(i)}$ to servers.

- 2: Both servers receive and store $c_j^{(i)}$.

Aggregate $(b, j, x, \{c_j^{(i)}\}_{i \in [N_c]})$: ▷ j -th message

- 1: Server b sets $\llbracket s \rrbracket_b \leftarrow \sum_i \Pi_{\text{SFSS}}.\text{Eval}(b, \text{k}_b^{(i)}, x, c_j^{(i)})$.
- 2: Return $\llbracket s \rrbracket_b$.

Figure 6: SFSS-based policy-hiding aggregation

the client-to-server communication in the sending phase (and hence the server-side storage). Notably, the total communication in the SFSS setup phase is independent of the number of messages per client, and each client only sets up one pair of SDPF keys for its data stream. As we will show later in §6, this reduces client-to-server communication and server-side storage significantly. A similar trend goes for (S)MDPF-based and (S)DCF-based phAgg. Additionally, the key for KH-PRF-based SFSS is larger than its non-streaming FSS key, as the KH-PRF-based SFSS key must additionally encode the KH-

Table 1: Communication Comparison of policy-hiding aggregation schemes. N_c denotes the number of data contributors; w denotes the number of messages from each contributor; $\mathbb{G}_{\text{in}}$ denotes the (S)FSS domain. For KH-PRF, we use LWR-based $F_{\text{LWR}}(\mathbf{k}, x) = \lfloor \langle H_1(x), \mathbf{k} \rangle \rfloor_p$ with $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^d$. For SMDPF, we use the concrete construction from §4.3.3. $C \rightarrow S$ denotes the total client-to-server communication, and $S \leftrightarrow S$ denotes the total inter-server communication. Highlighted cells denote the comparable better efficiency.

	Setup		Send		Aggregate	
	$C \rightarrow S$	$S \leftrightarrow S$	$C \rightarrow S$	$S \leftrightarrow S$	$C \rightarrow S$	$S \leftrightarrow S$
DPF-based [17]	No	No	$O(N_c \cdot w \cdot \lambda \cdot \log \mathbb{G}_{\text{in}})$	No	No	No
Vizard [24]	$O(N_c \cdot \lambda \cdot \log \mathbb{G}_{\text{in}})$	No	$O(N_c \cdot w)$	$O(N_c \cdot w)$	No	$\omega(1)$
SDPF-based	$O(N_c \cdot \lambda \cdot \log \mathbb{G}_{\text{in}})$	No	$O(N_c \cdot w)$	No	No	No
MDPF-based [17]	No	No	$O(t \cdot N_c \cdot w \cdot \lambda \cdot \log \mathbb{G}_{\text{in}})$	No	No	No
SMDPF-based	$O(t \cdot N_c \cdot (\lambda \cdot \log \mathbb{G}_{\text{in}} + d \log q))$	No	$O(N_c \cdot w)$	No	No	No
DCF-based [17]	No	No	$O(N_c \cdot w \cdot \lambda \cdot \log \mathbb{G}_{\text{in}})$	No	No	No
SDCF-based	$O(N_c \cdot (\lambda + d \log q) \cdot \log \mathbb{G}_{\text{in}})$	No	$O(N_c \cdot w)$	No	No	No

PRF key when necessary. For instance, an SMDPF key additionally encodes t KH-PRF keys following the instantiation from §4.3.3, and SDCF additionally encodes a KH-PRF key vector for each bit of input following the DCF construction from [17]. Nevertheless, each data stream only requires one single SFSS key, and each client sends a streaming ciphertext of constant size in each round, which saves significant communication in the message-sending phase than the FSS-based approach. We will report concrete comparison in §6.

Security. Fig. 6 securely realizes $\mathcal{F}_{\text{phAgg}}$. Correctness follows from SFSS, which ensures that any single SFSS key and streaming ciphertexts reveal no more information.

5.2 More SFSS-powered Applications

For attribute-hiding aggregation and window-based aggregation, please refer to Appendix §C.1 and §C.2, respectively. Here, we discuss conditional transciphering (CT) from SFSS, which may facilitate broader applications beyond aggregation.

Conditional transciphering. We can view SFSS as a CT primitive: an SFSS streaming ciphertext can be non-interactively converted to an additive sharing of its encrypted message only if certain conditions are met; otherwise, the servers share 0. CT can be applied beyond aggregation. Consider the following scenario: a lightweight client device (*e.g.*, a smartwatch) encrypts its data and uploads the ciphertexts to a storage server. At a later time, two computing servers want to perform a secure computation task over the stored ciphertexts. Before proceeding, they first transcipher the ciphertexts into additive shares suitable for subsequent secure processing. CT allows the transformation to occur only when predefined conditions are met, while hiding the filtering policy or attributes specified by the device. Also note that the transmission of SFSS ciphertexts does not require an additional secret channel, as the ciphertexts are pseudorandom by themselves. This is particularly appealing for applications where encrypted channels are hard or expensive to set up. This demonstrates that CT is not simply a tool for aggregation but a primitive with potential applications in a broad sense.

6 Performance Evaluation

This section reports the performance of SFSS and related policy-hiding aggregation. For attribute-hiding aggregation, please refer to Appendix C.1.

6.1 Implementation

We implement SFSS using C++ and open-source the code at <https://doi.org/10.5281/zenodo.17910080>. We conduct experiments on a PC equipped with Intel(R) Core(TM) i7-9750H CPU @ 2.6GHz and 64GB RAM, running Ubuntu 20.04. We simulate a local-area network (LAN, RTT: 0.2 ms,

10Gbps) and a wide-area network (WAN, RTT: 20 ms, 500Mbps) via Linux tc command. We set $\lambda = 128$.

6.2 SFSS Basic Primitives

Table 2 and Table 3 report the performance of SFSS compared to non-streaming FSS. We first compare SDPF with DPF. For predicate functions, we compare SDCF and DCF, with the LWR parameter setting in Table 6, Appendix §B.1.

Table 2: Performance comparison of generating/evaluating 1K instances of DPF, SDPF, DCF, and SDCF. We set $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{32}}$. $|\text{CTX}|$ denotes the ciphertext length. Key size, encryption time, and evaluation times are summed over 1K invocations. Note that key size and (Gen/Enc/Eval) running time are summed over 1K instances.

Scheme	Domain $\mathbb{G}_{\text{in}}$	Key Size (KB)	Gen (ms)	Enc (ms)	$ \text{CTX} $ (B)	Eval (ms)
DPF	$\mathbb{Z}_{2^{16}}$	309	25.0	-	4	0.5
	$\mathbb{Z}_{2^{32}}$	597	25.8	-	4	0.8
	$\mathbb{Z}_{2^{64}}$	1,173	27.5	-	4	1.7
SDPF	$\mathbb{Z}_{2^{16}}$	305	25.5	~ 0.1	4	0.5
	$\mathbb{Z}_{2^{32}}$	593	27.2	~ 0.1	4	0.9
	$\mathbb{Z}_{2^{64}}$	1,169	27.9	~ 0.1	4	1.8
DCF	$\mathbb{Z}_{2^{16}}$	369	27.6	-	4	0.9
	$\mathbb{Z}_{2^{32}}$	721	26.6	-	4	2.1
	$\mathbb{Z}_{2^{64}}$	1,425	27.7	-	4	3.0
SDCF	$\mathbb{Z}_{2^{16}}$	82,617	423.2	~ 7	4	162.9
	$\mathbb{Z}_{2^{32}}$	165,209	821.9	~ 7	4	328.4
	$\mathbb{Z}_{2^{64}}$	330,393	1,626.6	~ 7	4	660.3

Table 3: Performance comparison between DPF, SDPF, DCF, and SDCF with $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$ and different configurations of $\mathbb{G}_{\text{out}} \in \{\mathbb{Z}_{2^{20}}, \mathbb{Z}_{2^{40}}, \mathbb{Z}_{2^{60}}\}$. Key size and (Gen/Enc/Eval) running time are summed over 1K invocations.

$\mathbb{G}_{\text{out}}$	Scheme	Key Size (KB)	Gen (ms)	Enc (ms)	$ \text{CTX} $ (B)	Eval (ms)
$\mathbb{Z}_{2^{20}}$	DPF	1173	19.04	-	4	2.10
	SDPF	1,169	17.46	0.12	4	2.13
	DCF	1,425	21.70	-	4	3.86
	SDCF	206,745	1,045	4.21	4	411.98
$\mathbb{Z}_{2^{40}}$	DPF	1,177	17.70	-	8	2.09
	SDPF	1,169	18.00	0.13	8	2.13
	DCF	1,681	20.74	-	8	3.15
	SDCF	565,401	2,644.68	12.43	8	1,072.85
$\mathbb{Z}_{2^{60}}$	DPF	1,177	17.63	-	8	2.07
	SDPF	1,169	17.63	0.13	8	2.14
	DCF	1,681	17.18	-	8	3.37
	SDCF	1,210,521	7,600.13	28.77	8	2,455.08

Key and ciphertext size. Key and ciphertext size directly affect the communication and storage consumption in SFSS-based applications. All schemes exhibit roughly linear scaling in key size with respect to the bit size of $\mathbb{G}_{\text{in}}$. DPF and SDPF remain lightweight and scale efficiently, whereas DCF and SDCF show significantly steeper growth. This disparity arises

from the increased structural complexity of SDCF constructions. We use the DCF from [17] in our SDCF implementation. The DCF from [17] follows a tree structure during its key generation and evaluation, where each layer of the tree produces a correction word. When implementing this DCF for SDCF, we must embed a KH-PRF key at every layer of the tree using a correction word of size $|\mathbb{K}_{\text{kh}}|$, requiring $\log |\mathbb{G}_{\text{in}}|$ correction words in total. Since the LWR-based KH-PRF key is concretely large, the SDCF key is dominated by the correction words for encoding these large key vectors. For instance, with $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$ and $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{32}}$, SDCF requires encoding 64 correction words, each for the KH-PRF key $\mathbf{k} \in \mathbb{Z}_{2^{127}}^{320}$, which requires 320 KB and counts 97% of the total SDCF key size.

We note that SFSS achieves optimal constant ciphertext size. This property is particularly beneficial in streaming settings, where SFSS keys are not uploaded frequently, but streaming ciphertexts are transmitted continuously over time.

Running time. SDPF introduces negligible overhead compared to DPF. Across all different settings of $\mathbb{G}_{\text{in}}$, SDPF introduces only 0.1 ms for encrypting 1K messages while maintaining similar key and generation/evaluation time as DPF.

SDCF is a representative SFSS construction for predicate functions. SDCF incurs higher overhead compared to DCF for key generation and evaluation, due to the use of LWR-based KH-PRF. The SDCF key generation and evaluation times increase by more than two orders of magnitude compared to DCF. For example, when $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$, SDCF requires 1.6 s for generating 1K SDCF keys, and 660 ms for 1K invocations of SDCF evaluation. In contrast, DCF only requires 27.7 ms and 3 ms, respectively.

There are two main reasons for the efficiency gap. First, SDCF embeds the LWR-based KH-PRF key vector at every layer of the tree-based DCF. As a result, key generation and evaluation must repeatedly handle large vectors: expanding seeds, copying vectors in memory, and computing over them. These operations dominate the running time. In fact, they account for about 98% of the total SDCF cost in our measurements for $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$ and $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{30}}$. Second, SDPF benefits from hardware-accelerated PRF and PRG implementations in EMP-tool [58]. Our KH-PRF implementation currently does not support hardware acceleration. This lack of acceleration further widens the efficiency gap. We believe that adding hardware support for KH-PRF would narrow the efficiency gap, which is a promising direction for future engineering work.

Even though our implementation is not deeply optimized, SDCF is still practical for real-world applications: each SDCF requires ~ 330 KB of key size, ~ 1.7 ms for key generation, ~ 0.007 ms for streaming encryption, and ~ 0.6 ms for evaluation for $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$ and $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{32}}$. We stress that SDCF is not designed to outperform DCF in a single-instance setting. SDCF, like other SFSS schemes, targets the streaming setting. Its advantages appear when evaluating many messages over long-lived keys. SFSS properties such as optimal ciphertext size and efficient support for window-based aggregation make

SDCF-based applications significantly more attractive than DCF-based counterparts, as we will show in §6.3.2.

6.3 SFSS-based Policy-Hiding Aggregation

6.3.1 Equality-based Policy-hiding Aggregation

We can directly rely on SDPF to implement policy-hiding aggregations to support equality-based policy functions, as considered in Vizard [24]. We compare our SDPF-based policy-hiding aggregation with Vizard, in the scope of window-based aggregation that aggregates over a window range $[l, r]$ for each data stream (c.f., Appendix C.2). We conduct experiments with different window size and #contributors. Table 4 presents the communication and computation cost of SFSS-based phAgg, DPF-based non-streaming construction, and Vizard, with $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{32}}$, under LAN and WAN settings. Highlighted cells mean our solution achieves better efficiency.

Table 4: **Comparison of communication (MB) and running time (ms) for policy-hiding aggregation.** Note that Vizard requires secret-shared multiplication during aggregation; thus, we report its running time in the LAN (*i.e.*, Vizard (L)) and WAN (*i.e.*, Vizard (W)) settings, respectively.

Win. Size	Scheme	Number of Data Contributors					
		5K		10K		50K	
		Com.	Time	Com.	Time	Com.	Time
10	DPF-based	0	78.1	0	155.9	0	786.3
	Vizard (L)	≥ 16	15.3	≥ 16	26.9	≥ 16	97.4
	Vizard (W)	≥ 16	343.0	≥ 16	429.5	≥ 16	199.2
	Our	0	13.0	0	26.0	0	80.9
50	DPF-based	0	389.5	0	781.9	0	3866.4
	Vizard (L)	≥ 16	27.6	≥ 16	49.8	≥ 16	168.0
	Vizard (W)	≥ 16	128.9	≥ 16	152.8	≥ 16	279.3
	Our	0	13.5	0	27.1	0	87.2
100	DPF-based	0	797.4	0	1539.8	0	7755.1
	Vizard (L)	≥ 16	39.9	≥ 16	67.5	≥ 16	255.5
	Vizard (W)	≥ 16	140.5	≥ 16	172.3	≥ 16	361.3
	Our	0	14.5	0	27.9	0	90.4

Running time. The DPF-based policy-hiding aggregation does not support efficient window-based aggregation, as the server must perform DPF evaluation over each DPF key, which incurs DPF evaluation complexity that is linear in the product #contributors $\times$ #windows. Table 4 shows that the evaluation time is inefficient for a large number of contributors and a large window size, around $90\times$ that of SDPF-based construction. Both Vizard and our SDPF-based constructions support efficient window-based aggregation, where the servers only evaluate the DPF key once for each data stream. SDPF-based policy-hiding aggregation is more efficient than Vizard [24]. For instance, when the window size is 100 and the number of contributors is 10K, our SDPF-based construction only requires 14.5 ms both in LAN and WAN settings, while Vizard [24] requires 39.9 ms in LAN and 140.5 ms in WAN, *i.e.*, $2.7\times$ and $9\times$ slower, respectively.

Communication. Both DPF-based and SDPF-based policy-hiding aggregation do not require inter-server communication. Vizard [24] requires server-side secret-shared multiplication during aggregation. For $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{32}}$, Vizard requires at least 16 bytes of communication if all contributors have the same window size. However, when different contributors have different window sizes, Vizard will incur the worst communication overhead that is linear in the number of contributors; this is why we have ≥ 16 in Table 4, where 16 is the best efficiency we can hope for Vizard. SDPF-based policy-hiding aggregation does not require any inter-server communication.

Storage. Each server in our SDPF-based construction stores only one DPF key pair per stream, independent of the number of ciphertexts. In contrast, DPF-based baseline stores one key pair per message, leading to storage overhead linear in the total number of messages. Experimental results confirm this difference: DPF-based aggregation incurs $10 - 80\times$ higher storage than SDPF-based for $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{32}}$, and $10 - 40\times$ for $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{128}}$. Refer to Table 7 in Appendix §B.2 for details.

6.3.2 Solution from KH-PRF-based SFSS

Table 5 compares efficiency between DCF-based and SDCF-based policy-hiding aggregation. The main communication of the SDCF-based solution lies in transferring keys, with extremely small ciphertext-transfer communication, and the SDCF-based solution outperforms the DCF-based approach over communication and running time for large windows. This is because 1) SFSS enjoys an optimal streaming ciphertext size, which reduces communication significantly than the DCF-based approach. 2) SDCF-based only evaluates twice per data stream via window-based optimization, while the DCF-based approach must evaluate for each time slot.

Table 5: **Comparison of communication (MB) and running time (ms) for (S)DCF-based policy-hiding aggregation.** We set $\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{40}}$ and $\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{30}}$. Communication for SDCF-based contains offline key transfer and online ciphertext transfer. For example, (197, 7.6) denotes 197MB offline communication and 7.6MB online communication.

Win. Size	Scheme	Number of Data Contributors					
		10		100		1000	
		Com.	Time	Com.	Time	Com.	Time
10	DCF-based	<0.1	<1	0.1	1.3	1.3	12.8
	SDCF-based	(2, <0.001)	<1	(2, <0.01)	7.1	(197, <0.1)	70.3
100	DCF-based	<0.1	1.3	1.3	12.8	100.8	129.0
	SDCF-based	(2, <0.01)	<1	(20, <0.1)	7.3	(197, 0.8)	73.9
1000	DCF-based	1.3	12.8	12.9	125.8	1008.0	1257.2
	SDCF-based	(2, <0.1)	<1	(20, 0.8)	8.0	(197, 7.6)	84.2

7 Related Work

Function secret sharing. Gilboa and Ishai introduces the notion of distributed point function (DPF) [38]. Boyel, Gilboa,

and Ishai [17] extended DPF and formalized FSS, in which they also proposed a two-party DPF and distributed comparison function (DCF) with key size $O(\lambda \log n)$. Following [17], many optimizations [11, 18, 39] and extensions [16, 20, 21] are proposed on FSS itself. FSS has found broad applications in secure computation protocols and applications, including secure RAM and database applications [30, 31, 55, 57], pseudorandom correlation generator (PCG) [12, 13, 14, 15], mix-mode secure computation [11, 19], anonymous communication [29, 34], private set intersection [22, 36], and privacy-preserving machine learning [40, 41, 44, 52, 56].

Secure aggregation. Secure aggregation aggregates private data from different parties, which can be widely used in many secure computation protocols such as e-voting and federated learning. Secure aggregation can be constructed using generic secure multi-party computation [6, 48], dining cryptographers (DC) networks [26], pairwise additive masking [2, 7, 33], homomorphic or functional encryption [25, 27, 54]. Recently, FSS-based secure aggregation systems [9, 24, 28, 49] in the two-server distributed-trust model have gained attention due to their efficiency and simplicity, including Prio [28] for general statistics collection, Poplur [9] for heavy-hitter collection, Mastic [49] for attribute-based aggregation, and Vizard for policy-hiding aggregation [24]. In particular, FSS-based aggregation has been deployed by Mozilla [42] for browser data collection, Divvi Up [1] for private website analytics and surveys, and Apple and Google [4] for measuring the effectiveness of Covid-19 exposure notification. IETF is currently standardizing the FSS-based private aggregation [37]. Vizard is the only existing FSS-based work for streaming aggregation. As mentioned, Vizard cannot achieve its claimed security and efficiency properties simultaneously.

8 Conclusion

This paper introduces streaming function secret sharing, a new primitive tailored for secure computation over streaming data. We formalize the SFSS model, present concrete constructions, and demonstrate SFSS applications. Our SFSS-based solutions improve efficiency, strengthen security, and broaden functionality compared to existing approaches.

Limitations & future work. Our work leaves several interesting directions. First, a promising direction is *verifiable SFSS*, enabling verifiable streaming aggregation that resists malicious contributors. While zero-knowledge proofs for secret-shared data based on fully linear PCP [8, 28] offer a generic way to enforce well-formed SFSS keys, designing concretely efficient mechanisms that exploit SFSS structural properties remains an open challenge. Second, our current SFSS for predicate functions relies on LWR-based KH-PRF with a special zero-key, which contributes the main overhead for the resulting construction. It remains to design SFSS beyond single-point functions based on more concretely efficient primitives.

Acknowledgment

We thank the anonymous reviewers and shepherd for their insightful suggestions and comments. This work is supported by the National Research Foundation, Singapore, Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme and CyberSG R&D Cyber Research Programme Office, and National Natural Science Foundation of China under Grant 62302118. Any opinions, findings and conclusions or recommendations expressed in these materials are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, Cyber Security Agency of Singapore as well as CyberSG R&D Programme Office, Singapore.

Ethical Considerations

Stakeholder Analysis. Stakeholders include data contributors whose data is being aggregated, service providers who perform the aggregation, aggregation receivers who benefit from aggregated statistics, standardization groups such as IETF PPM WG, and privacy and security researchers. Data contributors benefit from improved privacy when their data is aggregated. Service providers can offer privacy-preserving services, enhancing user trust and compliance with privacy regulations. Aggregation receivers benefit from aggregated statistics without compromising individual privacy. Standardization groups can incorporate SFSS into guidelines for new extensions. Researchers may get inspired to perform future research for new primitives and applications.

Ethical Principles. We follow the four guiding principles outlined in the Menlo Report. We uphold *respect for persons* by ensuring individual data remains private during the aggregation. We promote *beneficence* by enabling service providers to derive useful insights without compromising individual privacy. We support *justice* by designing provably secure and efficient protocols, minimizing the resources required for real-world deployment. We adhere to *respect for law and public interest* by conducting all experiments on synthetic data.

Security Assumption, Possible Mismatch, and Mitigation. SFSS-based applications assume non-colluded servers. While this is a common assumption for FSS-based aggregation systems, there exists a possible extreme case where both servers are corrupted and/or colluded, and security collapses in this case. Therefore, any future applications based on our work must validate whether this assumption holds. Some possible scenarios: ① Two servers are separately run by two organizations without intentions to actively collude with each other due to conflicts of interest and/or regulations. ② One of the servers is run by an independent auditing organization, such as privacy regulation agencies. Protecting individual data is the ultimate goal of such agencies. ③ Design a game-theoretic incentive mechanism to discourage collusion among servers.

SFSS-based applications currently support semi-honest se-

curity; achieving a certain level of malicious security is future work. While the difference between semi-honest and malicious security is relatively well-known in the security and cryptography community, here we explicitly introduce the differences for the benefit of broader audiences, to prevent possible implementers from using SFSS with unmatched security assumptions. In particular, in a semi-honest secure protocol, the computing parties are assumed to follow the protocol specification, but they may try to learn additional information from the protocol execution. While for malicious security, corrupted parties may perform arbitrary deviations to undermine privacy and/or integrity.

Any applications attempting to achieve a certain level of malicious security when using SFSS should deploy additional mechanisms to check the well-formedness of the inputs from the users and/or correct execution of the protocol. In particular: ① The data contributors may be malicious, and they may submit malformed SFSS keys and/or ciphertexts to the servers. To mitigate this issue, the servers can ask the users to provide zero-knowledge proofs to prove the well-formedness of their inputs. ② The aggregation servers may be malicious, and they may deviate from the protocol specification to learn additional information about the users' inputs. To mitigate this issue, the servers can provide zero-knowledge proofs to prove that they have followed the protocol specification. For example, in secure aggregation, the servers can provide proofs to prove that they have correctly evaluated the SFSS ciphertexts and computed the final aggregation result.

Wellbeing for Team Members. We maintain open communication among team members, encourage sharing ideas and concerns, and provide support for any challenges faced during the research. While SFSS provides the first study of SFSS on security, efficiency, and functionality, with a promising connection to existing standardization, the authors may suffer from possible rejection, especially facing biased and/or irresponsible reviews, and misunderstandings. We support team members to defend authors' rights when applicable, maintain a healthy work-life balance, and seek support when needed.

Decision to Publish. We proceed with publication for the following reasons: ① *Advancing knowledge:* Publishing our research contributes knowledge for privacy-preserving technologies, providing valuable insights and techniques that benefit the broader research community and practitioners working on privacy and security. ② *Positive impact:* Our work has the potential to impact the design and implementation of secure aggregation systems and standardization efforts. By sharing our findings, we enable others to build upon our work and develop more secure and efficient solutions. ③ *Ethical considerations:* We have adhered to ethical research practices. We have also provided security considerations to guide future implementers. Overall, we believe that the benefits of publishing our research outweigh potential risks, and we are committed to promoting ethical standards in the dissemination and application of our work.

Open Science

The SFSS implementation we used for benchmarking can be found at <https://doi.org/10.5281/zenodo.17910080>. In the following, we provide an overview of our implementation and how to obtain the performance data in this paper.

Hardware requirement. SFSS currently supports 64-bit macOS and Ubuntu systems. Some cryptographic primitives, including pseudorandom generator (PRG), pseudorandom function (PRF), hash functions, require hardware acceleration (*i.e.*, SSE2 for x86_64 and ARM NEON). The hardware requirement is due to the involvement of emp-tool (<https://github.com/emp-toolkit/emp-tool>).

Software dependency. Our SFSS implementation depends on the following libraries:

- **CMake** (<https://cmake.org>). We use CMake (version ≥ 3.12) to build the code.
- **emp-tool** (<https://github.com/emp-toolkit/emp-tool> with tag 8052d95). We use the hardware-accelerated implementations of PRG, PRF, and hash functions from emp-tool.
- **GMP** (<https://gmplib.org>, version 6.3.0). We use `__uint128_t` to emulate modular operation over $\mathbb{Z}_{2^k}$ for $k < 128$. For $k \geq 128$, we use GMP and wrap our own big integer class `MyBigInteger` (in header file `field.h`).

Implementation overview. The project file structure is as follows:

```
SFSS/
├── CMakeLists.txt
├── util.h
├── twokeyprp.h
├── field.h
├── field.cpp
├── sfss.h
├── sharing.h
├── runner.h
├── sfss_main.cpp
├── client_runner_main.cpp
├── server_runner_main.cpp
└── README.md
```

Compile & Run Please refer to `README.MD` on how to compile the code and run benchmarks. To simulate WAN communication, we use `tc` command: `[sudo] tc qdisc add dev lo root netem delay 10ms rate 500mbit`.

Evaluation. Evaluating the benchmarking requires successfully compiling and generating the target executable files `sfss_main`, `client_runner_main`, and `server_runner_main`.

1) Basic primitives. The evaluation of basic primitive report efficiency comparison between DPF and SDPF, as well as DCF and SDCF. To obtain the results in Table 2 and Table 3. Simply put `bench_basic_sfss()` to the main func-

tion in `sfss_main.cpp`. Then open a terminal and work in the `SFSS/build` directory. Compile via `make` and run `./bin/sfss_main`. The evaluation results on key size and running time will be shown in the terminal.

2) Policy-hiding aggregation. To obtain the performance result on Table 4, do as follows:

2.1) Benchmarking Vizard. To obtain the benchmarking results for Vizard, do the following steps:

- Put `bench_vizard_main(argc, argv);` to the main function in `client_runner_main.cpp`; Put `bench_vizard_main(argc, argv);` to the main function in `server_runner_main.cpp`.
- Compile the project by running `make` in `SFSS/build`.
- Open three terminals. Run `./bin/client_runner_main` in terminal 1, `./bin/server_runner_main 0 12345` in terminal 2, and `./bin/server_runner_main 1 12346` in terminal 3.
- The performance results will be shown in the terminals.

2.2) Benchmarking DPF-based solution.

- Put `bench_dpf_window();` in the main function in `sfss_main.cpp`;
- Compile the project by running `make` in `SFSS/build`.
- Open a terminal and run `./bin/sfss_main` in a terminal.
- The performance results will be shown in the terminal.

2.3) Benchmarking SDPF-based solution. To obtain the benchmarking results for our SDPF-based policy-hiding aggregation, do the following steps:

- Put `bench_sdpf_main(argc, argv);` in the main function in `client_runner_main.cpp`; Put `bench_sdpf_main(argc, argv);` in the main function in `server_runner_main.cpp`.
- Compile the project by running `make` in `SFSS/build`.
- Open three terminals. Run `./bin/client_runner_main` in terminal 1, `./bin/server_runner_main 0 12345` in terminal 2, and `./bin/server_runner_main 1 12346` in terminal 3.
- The performance results will be shown in the terminals.

3) Attribute-hiding aggregation. To obtain the performance data, Do as follows:

- Put `bench_attribute_hiding();` in the main function in `sfss_main.cpp`.
- Compile the project by running `make` in `SFSS/build`.
- Open a terminal and run `./bin/sfss_main` in the terminal.
- The performance results will be shown in the terminal.

References

- [1] About divvi up. <https://divviup.org/about/#open-source>. Accessed: 2025-08-09.
- [2] Gergely Ács and Claude Castelluccia. I have a dream!(differentially private smart metering). In *International Workshop on Information Hiding*, pages 118–132. Springer, 2011.
- [3] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.
- [4] Apple and Google. Exposure notification privacy-preserving analytics (enpa) white paper. 2021.
- [5] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 719–737. Springer, 2012.
- [6] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*, pages 351–371. 2019.
- [7] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.
- [8] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear pcps. In *Annual international cryptology conference*, pages 67–97. Springer, 2019.
- [9] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 762–776. IEEE, 2021.
- [10] Dan Boneh, Kevin Lewi, Hart Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *Annual Cryptology Conference*, pages 410–428. Springer, 2013.
- [11] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 871–900. Springer, 2021.
- [12] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector ole. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 896–912, 2018.
- [13] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round ot extension and silent non-interactive secure computation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 291–308, 2019.
- [14] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent ot extension and more. In *Annual International Cryptology Conference*, pages 489–518. Springer, 2019.
- [15] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-lpn. In *Annual International Cryptology Conference*, pages 387–416. Springer, 2020.
- [16] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Yaxin Tu. Improved constructions for distributed multi-point functions. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2414–2432. IEEE, 2025.
- [17] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367. Springer, 2015.
- [18] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1292–1303, 2016.
- [19] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In *Theory of Cryptography: 17th International Conference, TCC 2019, Nuremberg, Germany, December 1–5, 2019, Proceedings, Part I 17*, pages 341–371. Springer, 2019.
- [20] Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I Kolobov. Programmable distributed point functions. In *Annual International Cryptology Conference*, pages 121–151. Springer, 2022.
- [21] Elette Boyle, Lisa Kohl, Zhe Li, and Peter Scholl. Direct fss constructions for branching programs and more

- from prgs with encoded-output homomorphism. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 266–298. Springer, 2024.
- [22] Dung Bui, Gayathri Garimella, Peihan Miao, and Phuoc Van Long Pham. New framework for structure-aware psi from distributed function secret sharing. *Cryptology ePrint Archive*, 2025.
- [23] Lukas Burkhalter, Anwar Hithnawi, Alexander Viand, Hossein Shafagh, and Sylvia Ratnasamy. {TimeCrypt}: Encrypted data stream processing at scale with cryptographic access control. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 835–850, 2020.
- [24] Chengjun Cai, Yichen Zang, Cong Wang, Xiaohua Jia, and Qian Wang. Vizard: A metadata-hiding data analytic system with end-to-end policy controls. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 441–454, 2022.
- [25] T-H Hubert Chan, Elaine Shi, and Dawn Song. Privacy-preserving stream aggregation with fault tolerance. In *International Conference on Financial Cryptography and Data Security*, pages 200–214. Springer, 2012.
- [26] David Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of cryptography*, 1(1):65–75, 1988.
- [27] Jérémy Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Decentralized multi-client functional encryption for inner product. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 703–732. Springer, 2018.
- [28] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *14th USENIX symposium on networked systems design and implementation (NSDI 17)*, pages 259–282, 2017.
- [29] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE Symposium on Security and Privacy*, pages 321–338. IEEE, 2015.
- [30] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. Waldo: A private time-series database from function secret sharing. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2450–2468. IEEE, 2022.
- [31] Jack Doerner and Abhi Shelat. Scaling oram for secure computation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 523–535, 2017.
- [32] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [33] Tariq Elahi, George Danezis, and Ian Goldberg. Privex: Private collection of traffic statistics for anonymous communication networks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 1068–1079, 2014.
- [34] Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In *30th USENIX Security Symposium*, pages 1775–1792, 2021.
- [35] Pierre-Alain Fouque, Paul Kirchner, Thomas Pornin, and Yang Yu. BAT: Small and Fast KEM over NTRU Lattices. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022:240–265, 02 2022.
- [36] Gayathri Garimella, Mike Rosulek, and Jaspal Singh. Structure-aware private set intersection, with applications to fuzzy matching. In *Annual International Cryptology Conference*, pages 323–352. Springer, 2022.
- [37] Tim Geoghegan, Christopher Patton, Eric Rescorla, and Christopher A Wood. Distributed aggregation protocol for privacy preserving measurement. *IETF*, 2023.
- [38] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 640–658. Springer, 2014.
- [39] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in cot and dpf. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 330–362. Springer, 2023.
- [40] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive*, 2023.
- [41] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. Llama: A low latency math library for secure inference. *Cryptology ePrint Archive*, 2022.
- [42] Bobby Holley. Built for privacy: Partnering to deploy oblivious http and prio in firefox. <https://blog.mozilla.org/en/firefox/partnership-ohhttp-prio>. Accessed: 2025-08-09.

- [43] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jian-sheng Ding. Cheetah: Lean and fast secure {Two-Party} deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.
- [44] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. Orca: Fss-based secure training and inference with gpus. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 597–616. IEEE, 2024.
- [45] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Mascot: faster malicious arithmetic secure computation with oblivious transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 830–842, 2016.
- [46] Tung Le, Rouzbeh Behnia, Jorge Guajardo, and Thang Hoang. {MUSES}: Efficient {Multi-User} searchable encrypted database. In *33rd USENIX Security Symposium*, pages 2581–2598, 2024.
- [47] MATZOV. Report on the Security of LWE: Improved Dual Lattice Attack. 2022.
- [48] Silvio Micali, Oded Goldreich, and Avi Wigderson. How to play any mental game. In *Proceedings of the Nineteenth ACM Symp. on Theory of Computing, STOC*, pages 218–229. ACM New York, 1987.
- [49] Dimitris Mouris, Christopher Patton, Hannah Davis, Pratik Sarkar, and Nektarios Georgios Tsoutsos. Mastic: Private weighted heavy-hitters and attribute-based metrics. *Proceedings on Privacy Enhancing Technologies*, 2025.
- [50] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and kdc. In *International conference on the theory and applications of cryptographic techniques*, pages 327–346. Springer, 1999.
- [51] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-Party Secure Inference. In *CCS*, pages 325–342, 2020.
- [52] Théo Ryffel, Pierre Tholoni, David Pointcheval, and Francis Bach. Ariann: Low-interaction privacy-preserving deep learning via function secret sharing. *arXiv preprint arXiv:2006.04593*, 2020.
- [53] Kazue Sako and Joe Kilian. Secure voting using partially compatible homomorphisms. In *Annual international cryptology conference*, pages 411–424. Springer, 1994.
- [54] Elaine Shi, HTH Chan, Eleanor Rieffel, Richard Chow, and Dawn Song. Privacy-preserving aggregation of time-series data. In *Annual Network & Distributed System Security Symposium (NDSS)*. Internet Society, 2011.
- [55] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duo-ram: A {Bandwidth-Efficient} distributed {ORAM} for 2-and 3-party computation. In *32nd USENIX Security Symposium*, pages 3907–3924, 2023.
- [56] Sameer Wagh. Pika: Secure computation using function secret sharing over rings. *Proceedings on Privacy Enhancing Technologies*, 2022.
- [57] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, 2017.
- [58] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.

A Additional Constructions

A.1 SDPF with Weighted Output

Fig. 7 shows the construction for single-point functions with weighted output. The only difference is that the streaming encryption computes the streaming ciphertext as $c_{\text{ctr}} \leftarrow m - (F(\llbracket r \rrbracket_0, \text{ctr}) - F(-\llbracket r \rrbracket_1, \text{ctr})) \cdot \beta^{-1}$, which requires $\mathbb{G}_{\text{out}} = \mathbb{F}$ for some finite field to support the inverse β^{-1} . The correctness and security analysis are essentially the same as in §4.2.

A.2 Error Reduction & Removal

Our starting point is a simple and efficient *scale-then-truncate* approach [46] to reduce the error to at most one bit. Specifically, we modify the SFSS encryption algorithm to scale the message m as

$$m' = m \cdot 2^{\lceil \log(B+1) \rceil},$$

where B is the upper bound on the error. To reduce the error to only 1 bit, it is sufficient to truncate each evaluation share by $\lceil \log(B+1) \rceil$ bits and then add 1 to the truncated shared secret. This approach only leaves at most 1 bit of error from the truncation error. We can extend the idea to the multiple-client case. Suppose n clients contribute SFSS keys, and the i -th client encodes its message m_i as

$$m'^{(i)} = m_i \cdot 2^{\lceil \log(nB+1) \rceil}.$$

By the definition of SFSS evaluation, the parties can compute the aggregate

$$\sum_i m'^{(i)} + \sum_i e^{(i)} = \sum_i m_i \cdot 2^{\lceil \log(nB+1) \rceil} + \sum_i e^{(i)},$$

Parameters: PRF $F : \{0, 1\}^\lambda \times \mathbb{G}_{\text{ctr}} \rightarrow \mathbb{G}_{\text{out}}$; Function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda}$ of point functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$; Function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda}$ of point functions with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda$; DPF scheme Π_{DPF} for function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda}$; We require $\mathbb{G}_{\text{out}} = \mathbb{F}$ for a finite field $\mathbb{F}$, and $|\mathbb{G}_{\text{ctr}}| \geq 2^\lambda$.
 $\text{Gen}(1^\lambda, \hat{f}_{\alpha, \beta} \in \mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}})$:

- 1: Set $\text{ctr} = 1$
- 2: $r \xleftarrow{\$} \{0, 1\}^\lambda, (k_0^{\text{dpf}}, k_1^{\text{dpf}}) \leftarrow \Pi_{\text{DPF}}.\text{Gen}(1^\lambda, \hat{f}_{\alpha, (\beta, r)})$
- 3: For $b \in \{0, 1\}$, $(\llbracket \beta \rrbracket_b, \llbracket r \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{\text{dpf}}, \alpha)$
- 4: $\text{st}_f \leftarrow \{\text{ctr}\}, k_e \leftarrow \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}, k_0 \leftarrow \{k_0^{\text{dpf}}\}, k_1 \leftarrow \{k_1^{\text{dpf}}\}$
- 5: Return (st_f, k_0, k_1)

$\text{Enc}(\text{st}_f, k_e, m)$:

- 1: Parse $\text{st}_f = \{\text{ctr}\}$ and $k_e = \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}$
- 2: $c_{\text{ctr}} \leftarrow m - (F(\llbracket r \rrbracket_0, \text{ctr}) - F(-\llbracket r \rrbracket_1, \text{ctr})) \cdot \beta^{-1}$
- 3: Set $c \leftarrow (\text{ctr}, c_{\text{ctr}})$ and update $\text{st}_f \leftarrow \{\text{ctr} + 1\}$
- 4: Return (st_f, c)

$\text{Eval}(b, k_b, x, c)$:

- 1: Parse $c = (j, c_j)$ and $k_b = \{k_b^{\text{dpf}}\}$
- 2: $(\llbracket e \rrbracket_b, \llbracket h \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{\text{dpf}}, x)$
- 3: $\llbracket s \rrbracket_b \leftarrow \llbracket e \rrbracket_b \cdot c_j + (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, j)$
- 4: Return $\llbracket s \rrbracket_b$

Figure 7: The Proposed SDPF construction for $\beta \geq 1$

Parameters: $\llbracket x \rrbracket$ with $x \in \mathbb{Z}_{2^\ell}$; truncation bits f ; 2PC compare protocol $\llbracket \mathbf{1}\{x > y\} \rrbracket^B \leftarrow \text{CMP}(x, y)$; Bit to Arithmetic conversion protocol $\llbracket x \rrbracket \leftarrow \text{B2A}(\llbracket x \rrbracket^B)$.

Wrap-around Error:

- 1: S_0 and S_1 call CMP protocol: $(\llbracket w \rrbracket_0^B, \llbracket w \rrbracket_1^B) \leftarrow \text{CMP}(\llbracket x \rrbracket_0, 2^\ell - 1 - \llbracket x \rrbracket_1)$
- 2: Two servers invoke B2A protocol: $(\llbracket w \rrbracket_0^f, \llbracket w \rrbracket_1^f) \leftarrow \text{B2A}(\llbracket w \rrbracket_0^B, \llbracket w \rrbracket_1^B)$

Least bit Error:

- 3: S_b extracts the least f bits of $\llbracket x \rrbracket_b$ as u_b locally
- 4: S_0 and S_1 call CMP protocol: $(\llbracket c \rrbracket_0^B, \llbracket c \rrbracket_1^B) \leftarrow \text{CMP}(u_0, 2^f - 1 - u_1)$
- 5: Two servers invoke B2A protocol: $(\llbracket c \rrbracket_0, \llbracket c \rrbracket_1) \leftarrow \text{B2A}(\llbracket c \rrbracket_0^B, \llbracket c \rrbracket_1^B)$
- 6: S_b sets $\llbracket z \rrbracket_b = (\llbracket x \rrbracket_b \ggg f) - \llbracket w \rrbracket_b^f \cdot 2^{\ell-f} + \llbracket c \rrbracket_b$
- 7: Return $\llbracket z \rrbracket_b$

Figure 8: Faithful Two-Party Truncation.

where each $e^{(i)} \in [B]$ is the bounded error introduced by the i -th evaluation. Since $\sum_i e^{(i)} \leq n \cdot B$, it affects only the least $\lceil \log(nB + 1) \rceil$ bits of $\sum_i m^{(i)}$, thus the error does not interfere with the message bits. Each evaluator can locally truncate the least significant of $\lceil \log(nB + 1) \rceil$ bits, which reduces the error to at most 1 bit (due to the truncation error).

Taking a step further, we can totally remove the truncation error via 2-party computation (2PC). Since the error affects only the least significant $\lceil \log(B + 1) \rceil$ bits, the parties can engage in a 2PC faithful truncation protocol to jointly extract and correct these bits, thereby recovering the exact result. Note that this error-elimination step incurs only constant communication, regardless of the number of participating clients. Similarly, if the results are of ℓ bits, the communication cost is of $12(\ell + \lceil \log n + \log B \rceil)$ bits in $\log_2 \ell + 1$ rounds; If the results are of $\ell - \lceil \log n + \log B \rceil$ bits, the communication is of $12\lceil \log n + \log B \rceil + \ell$ bits in $\log \lceil \log n + \log B \rceil + 1$ rounds.

Given an ℓ -bit integer x and its arithmetic shares $(\llbracket x \rrbracket_0, \llbracket x \rrbracket_1)$, Fig. 8 presents the 2PC protocol for its faithful truncation following [43, 51]. First, w denotes whether the sum of two shares wraps around modulo 2^ℓ . If so, there will be a large error due to local truncation, and it should be corrected by subtracting 2^ℓ . c comes from the last-bit error that is decided by whether the sum of the least significant f bits of the two shares exceeds 2^f . Let $w = \mathbf{1}\{\llbracket x \rrbracket_0 + \llbracket x \rrbracket_1 > 2^\ell - 1\}$, u_b denotes the last f bits of $\llbracket x \rrbracket_b$ and $c = \mathbf{1}\{u_0 + u_1 > 2^f - 1\}$, we have $(\llbracket x \rrbracket_0 \ggg f) + (\llbracket x \rrbracket_1 \ggg f) - w \cdot 2^\ell + c = (x \ggg f)$. By using the VOLE style-OT, the communication complexity is $12\ell + 12f$ bits in $\log_2 \ell + 1$ rounds.

A.3 LHE-based Approach

We give a concrete “FSS+LHE” construction using the exponential ElGamal encryption [53] that supports additive homomorphism in the exponent.

In particular, the servers jointly generate a public key $h = g^{\text{sk}}$, where sk is the secret key additively shared between the servers and g is the generator for a DDH-hard cyclic group $\mathbb{G}$ with group order p . Suppose we have an FSS scheme that can be used to additively share $f(x)$ over $\mathbb{Z}_p$. With a ciphertext

$$c = (c_1, c_2) = (g^r, h^r \cdot g^m),$$

servers compute as follows:

1. For $b \in \{0, 1\}$, server b samples a random share $\llbracket \gamma \rrbracket_b \leftarrow \mathbb{Z}_p$, computes

$$c'_{1,b} \leftarrow g^{\llbracket \gamma \rrbracket_b} \cdot c_1^{f(x)_b}, c'_{2,b} \leftarrow h^{\llbracket \gamma \rrbracket_b} \cdot c_2^{f(x)_b},$$

and sends $(c'_{1,b}, c'_{2,b})$ to server $1 - b$.

2. For $b \in \{0, 1\}$, server b computes

$$c' = (c'_1, c'_2) \leftarrow (c'_{1,0} \cdot c'_{1,1}, c'_{2,0} \cdot c'_{2,1}).$$

3. Server 0 defines $f_0 \leftarrow (c'_1)^{-\llbracket \text{sk} \rrbracket_0}$, and server 1 defines $f_1 \leftarrow c'_2 \cdot (c'_1)^{-\llbracket \text{sk} \rrbracket_1}$.

Parameters: A function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$; The bit-decomposed function family of $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$ denoted as $\mathcal{H}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}^\ell}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}^\ell$; An SFSS scheme Π_{SFSS} for the predicate function family $\mathcal{H}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}^\ell$;

$\text{Gen}(1^\lambda, \hat{f} \in \mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}})$:

- 1: Let $h \in \mathcal{H}$ be the output-decomposed function of f
- 2: $(\text{st}_h^{\text{sfss}}, k_e^{\text{sfss}}, k_0^{\text{sfss}}, k_1^{\text{sfss}}) \leftarrow \Pi_{\text{SFSS}}.\text{Gen}(1^\lambda, \hat{h})$
- 3: $\text{st}_f \leftarrow \{\text{st}_h^{\text{sfss}}\}, k_e \leftarrow \{k_e^{\text{sfss}}\}, k_0 \leftarrow \{k_0^{\text{sfss}}\}, k_1 \leftarrow \{k_1^{\text{sfss}}\}$
- 4: Return (st_f, k_0, k_1)

$\text{Enc}(\text{st}_f, k_e, m)$:

- 1: Parse $\text{st}_f = \{\text{st}_h^{\text{sfss}}\}$ and $k_e = \{k_e^{\text{sfss}}\}$
- 2: $(\text{st}_f^{\text{sfss}}, c) \leftarrow \Pi_{\text{SFSS}}.\text{Enc}(\text{st}_f^{\text{sfss}}, k_e^{\text{sfss}}, m)$
- 3: Update $\text{st}_f = \{\text{st}_h^{\text{sfss}}\}$
- 4: Return (st_f, c)

$\text{Eval}(b, k_b, x, c)$:

- 1: Parse $k_b = \{k_b^{\text{sfss}}\}$
- 2: $\llbracket s \rrbracket_b \leftarrow \Pi_{\text{SFSS}}.\text{Eval}(b, k_b^{\text{sfss}}, x, c)$
- 3: Return $\llbracket s \rrbracket_b = (\llbracket s_{\ell-1} \rrbracket, \dots, \llbracket s_1 \rrbracket, \llbracket s_0 \rrbracket)$.

Figure 9: The SFSS framework for generic functions.

We can check that $f_0 \cdot f_1 = g^{f(x) \cdot m}$, meaning that f_0 and f_1 multiplicatively share $g^{f(x) \cdot m}$ (equivalently, additive sharing of $f(x) \cdot m$ in the exponent). Note that the parties can reconstruct $g^{f(x) \cdot m}$ by disclosing shares f_0 and f_1 to each other. When $f(x) \cdot m$ is small enough, the result $f(x) \cdot m$ can be recovered by exhaustive search.

The above protocol is essentially a classic secret-shared decryption protocol for the ElGamal encryption scheme. Correctness is easy to check:

$$c'_{1,b} \leftarrow g^{\llbracket y \rrbracket_b + r \cdot \llbracket f(x) \rrbracket_b}, c'_{2,b} \leftarrow h^{\llbracket y \rrbracket_b + r \cdot \llbracket f(x) \rrbracket_b} \cdot g^{m \cdot \llbracket f(x) \rrbracket_b},$$

$$c'_1 = c'_{1,0} \cdot c'_{1,1} = g^{\gamma + r \cdot f(x)}, c'_2 = c'_{2,0} \cdot c'_{2,1} = h^{\gamma + r \cdot f(x)} \cdot g^{m \cdot f(x)}.$$

Thus, $c' = (c'_1, c'_2)$ is indeed a ciphertext of $f(x) \cdot m$ in the exponent. As follows,

$$f_0 \cdot f_1 = (c'_1)^{-\llbracket \text{sk} \rrbracket_0} \cdot c'_2 \cdot (c'_1)^{-\llbracket \text{sk} \rrbracket_1} = g^{m \cdot f(x)}.$$

As for privacy, $g^{\llbracket y \rrbracket_b}$ and $h^{\llbracket y \rrbracket_b}$ are used to randomize $(c'_1)^{\llbracket f(x) \rrbracket_b}, c'_2)^{\llbracket f(x) \rrbracket_b}$, so that nothing about $\llbracket f(x) \rrbracket_b$ is revealed; this is a standard randomization trick.

A.4 SFSS for Generic Functions

For the generic case with function outputs not limited to predicate values (0/1), we design a generic SFSS framework via a “predicate decomposition” trick.

Tool: bit decomposition. Given any $x \in \mathbb{G}$, one can decompose x into a vector of values $(x_{\ell-1}, \dots, x_1, x_0) \in \mathbb{G}^\ell$ such

that $x = \sum_{i \in [\ell]} g^i \cdot x_i$ for some base $g \in \mathbb{G}$. Following existing notations [45], we use a gadget vector

$$g = (1, g^1, g^2, \dots, g^{\ell-1}) \in \mathbb{G}^\ell.$$

$g^{-1} : \mathbb{G} \rightarrow \mathbb{G}^\ell$ denotes the bit decomposition function that maps $x \in \mathbb{G}$ into a *binary* vector $\mathbf{x} = (x_{\ell-1}, \dots, x_1, x_0) = g^{-1}(x) \in \mathbb{G}^\ell$. We can map $\mathbf{x}$ back to x by inner-product $x = \langle \mathbf{g}, \mathbf{x} \rangle$. The base g depends on $\mathbb{G}$. For example, we can have $g = 2$ for $\mathbb{G} = \mathbb{F}_p$ and $g = X$ for $\mathbb{G} = \mathbb{F}_{2^\ell}$.

Predicate decomposition. Our framework follows a simple idea: any function output $f(x)$ can be decomposed into a binary form $\mathbf{y} = (y_0, y_1, \dots, y_{\ell-1}) \leftarrow g^{-1}(f(x))$. To compute $m \cdot y$, it’s sufficient to compute

$$m \cdot \langle \mathbf{g}, \mathbf{y} \rangle = \sum_i m \cdot g_i \cdot y_i = \langle \mathbf{g}, m \cdot \mathbf{y} \rangle.$$

Therefore, we can rely on SFSS for predicate functions to compute each $y_i \cdot m$ separately, and then linearly combine the bit-wise outputs using $\mathbf{g}$.

Correctness follows from the construction and the correctness of SFSS for predicate functions. For any $x \in \mathbb{G}_{\text{in}}$ such that $y = f(x) = \langle \mathbf{g}, g^{-1}(f(x)) \rangle$, we have $\llbracket s_i \rrbracket_0 + \llbracket s_i \rrbracket_1 = y_i \cdot m + e_i$, where $\mathbf{e} \in [0, B]^\ell$ is an error vector with bounded ℓ_1 -norm introduced by the KH-PRF. Overall, we have $\mathbf{s} = m \cdot \mathbf{y} + \mathbf{e} = m \cdot \text{BitDecom}(f(x)) + \mathbf{e}$ as required. The servers can use the error reduction/elimination method in §4.3 and Appendix §A.2 to remove these errors before conducting the remaining linear combination to share $m \cdot f(x)$.

Security of the generic SFSS construction follows directly from the security of SFSS for predicate functions, as the only additional step is decomposing function outputs into binary form. And no further leakage is introduced. Thus, we do not present the same proof again.

Remark 1. One may be concerned about the feasibility of SFSS construction for general functions, because this SFSS framework requires an existing FSS scheme for $\mathcal{F}$, which can be difficult to realize for generic functions [17, 18] based on well-studied assumptions. Moreover, bit-decomposition for every output $f(x)$ could be computationally infeasible for large domains. Nevertheless, this SFSS framework is cheap to realize for many useful functions, including multi-point functions and comparison functions with weighted outputs, where efficient FSS for these functions are already available, and the output bit-decomposition can be done efficiently. Moreover, these functions are the focus of existing FSS works, facilitating a range of applications. Our SFSS framework provides a universal compiler to convert these FSS schemes to their streaming variants efficiently.

B Additional Implementation & Performance

B.1 LWR-based KH-PRF Parameter Setting

The goal of LWR parameter setting is to determine d and q in the LWR-based KH-PRF $F_{\text{LWR}}(\mathbf{k}, x) : \mathbb{Z}_q^d \times \{0, 1\}^* \rightarrow \mathbb{Z}_p$ according to p , where p depends on application settings. We adopt a parameter selection strategy that maximizes efficiency while maintaining security guarantees. For concrete security estimation, we follow the standard methodology widely adopted in lattice-based cryptography literature. Our security analysis relies on current analyses of the concrete hardness of LWR, which indicate that no known attack method provides an advantage against LWR compared to *Learning-With-Error* (LWE).

Table 6: Parameter setting for LWR-based KH-PRF.

p	2^{10}	2^{20}	2^{30}	2^{40}	2^{50}	2^{60}
d	200	200	320	550	840	1180
q	2^{30}	2^{100}	2^{127}	2^{127}	2^{127}	2^{127}

Following the approach established in prior works such as Saber [32] and BAT [35], we estimate the security level of our LWR-based construction to be equivalent to that of LWE, using the lattice estimator¹ from [3] with the cost model from [47]. In practice, we observe that the bit length of the modulus q has a minor impact on computational efficiency when it remains below 128 bits because we use `__uint128_t` in our implementation to emulate the modular arithmetic over $\mathbb{Z}_q$ in this case, while efficiency is highly sensitive to the lattice dimension d . Therefore, targeting 128-bit classical security, we select parameters that minimize d to achieve optimal efficiency in running time, as detailed in Table 6.

B.2 Storage for Policy-hiding Aggregation

Table 7 shows detailed storage comparison between DPF-based, Vizard, and SDPF-based policy-hiding aggregation. The SDPF-based approach achieves the optimal server-side storage as Vizard, yet significantly outperforms the non-streaming DPF-based approach. As mentioned, the reason is that SFSS-based solutions only requires one single ciphertext for each streaming message, while the FSS-based approach encodes a streaming message together with a policy function using a pair of FSS keys, which is concretely high for a large G_{in} .

¹<https://github.com/malb/lattice-estimator>

Parameters: A SFSS scheme Π_{SFSS} for a function family $\mathcal{F}_{G_{\text{in}}, G_{\text{out}}}$ with domain G_{in} and range G_{out} ; N_a denotes the number of attributes; ℓ_i denotes the bit size of i -th attribute, $\ell = \sum_{i \in [N_a]} \ell_i$, and $\log |G_{\text{in}}| = \ell$;

Setup $(1^\lambda, (\alpha_1, \alpha_2, \dots, \alpha_{N_a}); \perp; \perp)$:

- 1: The client i defines $\alpha = \alpha_1 || \alpha_2 || \dots || \alpha_{N_a}$ and a point function $f_{\alpha, 1}$. The client i generates SDPF keys

$$(\text{st}_f, k_0^{(i)}, k_1^{(i)}) \leftarrow \Pi_{\text{SDPF}}.\text{Gen}(1^\lambda, \hat{f}_{\alpha, 1}),$$

and sends $k_b^{(i)}$ to the server b for $b \in \{0, 1\}$.

- 2: Client i stores $\text{st}^{(i)}$ locally. Server b stores $k_b^{(i)}$.

Send $((\text{st}_f, k_e^{(i)}, m_j^{(i)}); \perp; \perp)$:

- 1: Client i encrypt its j -th streaming message $m_j^{(i)}$

$$(\text{st}_f^{(i)}, c_j^{(i)}) \leftarrow \Pi_{\text{SDPF}}.\text{Enc}(\text{st}_f^{(i)}, k_e^{(i)}, m_j^{(i)}).$$

- 2: Sends $c_j^{(i)}$ to both servers.

Aggregate $(b, j, p, \{c_j^{(i)}\}_{i \in [N_c]})$:

- 1: Define the accepting set

$$\mathcal{X} = \{x | \forall x \in \{0, 1\}^\ell \text{ s.t. } p(x_1, x_2, \dots, x_{N_a}) = 1\}.$$

- 2: Server b computes

$$[s]_b \leftarrow \sum_{i \in [N_c]} \sum_{x \in \mathcal{X}} \Pi_{\text{SDPF}}.\text{Eval}(b, k_b^{(i)}, x, c_j^{(i)}).$$

Figure 10: SDPF-based attribute-hiding aggregation.

C Additional Applications

C.1 Attribute-hiding Aggregation

Generic method from 2PC. We can resort to two-party computation (2PC) for secure attribute-hiding aggregation. In this approach, each client shares its attributes and data items with the servers. To perform attribute-based aggregation, the servers run a 2PC protocol to securely evaluate the policy function over the shared attributes and include the data item in the output only if the attributes satisfy the predicate. However, this method incurs communication that is (super-)linear in the number of clients and requires multiple rounds of interaction between the servers, especially for complex predicates.

In contrast, we propose a simple attribute-encoding method via single-point functions. This enables a solution using SDPF that supports *non-interactive aggregation*, requiring no communication between the servers during evaluation.

High-level idea. Suppose a client has a message m and N_a

Table 7: Comparison of storage consumption (MB) of different schemes.

Message Space	#Ciphertexts Per User	Scheme	$\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{32}}$				$\mathbb{G}_{\text{in}} = \mathbb{Z}_{2^{64}}$			
			Number of contributors				Number of contributors			
			100	1000	10000	100000	100	1000	10000	100000
$\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{32}}$	10	DPF-based	0.57	5.69	56.93	569.34	1.12	11.19	111.87	1118.66
		Vizard [24]	0.06	0.65	6.46	64.56	0.12	1.19	11.95	119.50
		Our	0.06	0.64	6.42	64.18	0.12	1.19	11.91	119.11
	100	DPF-based	5.69	56.93	569.34	5693.44	11.19	111.87	1118.66	11186.60
		Vizard [24]	0.13	1.33	13.32	133.23	0.19	1.88	18.82	188.16
		Our	0.13	1.33	13.28	132.85	0.19	1.88	18.78	187.78
	1000	DPF-based	56.93	569.34	5693.44	56934.36	111.87	1118.66	11186.60	111866.00
		Vizard [24]	0.82	8.20	81.99	819.87	0.87	8.75	87.48	874.81
		Our	0.82	8.19	81.95	819.49	0.87	8.74	87.44	874.42
$\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{64}}$	10	DPF	0.59	5.88	58.84	588.42	1.12	11.22	112.25	1122.47
		Vizard [24]	0.07	0.73	7.26	72.57	0.13	1.28	12.75	127.51
		Our	0.07	0.72	7.18	71.81	0.13	1.27	12.67	126.74
	100	DPF-based	5.88	58.84	588.42	5884.17	11.22	112.25	1122.47	11224.75
		Vizard [24]	0.21	2.10	20.99	209.90	0.26	2.65	26.48	264.84
		Our	0.21	2.09	20.91	209.14	0.26	2.64	26.41	264.07
	1000	DPF-based	58.84	588.42	5884.17	58841.71	112.25	1122.47	11224.75	112247.47
		Vizard [24]	1.58	15.83	158.32	1583.19	1.64	16.38	163.81	1638.13
		Our	1.58	15.82	158.24	1582.43	1.64	16.37	163.74	1637.36
$\mathbb{G}_{\text{out}} = \mathbb{Z}_{2^{128}}$	10	DPF-based	0.58	5.81	58.08	580.79	1.13	11.30	113.01	1130.10
		Vizard [24]	0.09	0.89	8.86	88.60	0.14	1.44	14.35	143.53
		Our	0.09	0.87	8.71	87.07	0.14	1.42	14.20	142.00
	100	DPF-based	5.81	58.08	580.79	5807.88	11.30	113.01	1130.10	11301.04
		Vizard [24]	0.36	3.63	36.33	363.25	0.42	4.18	41.82	418.19
		Our	0.36	3.62	36.17	361.73	0.42	4.17	41.67	416.66
	1000	DPF-based	58.08	580.79	5807.88	58078.77	113.01	1130.10	11301.04	113010.41
		Vizard [24]	3.11	31.10	310.98	3109.84	3.16	31.65	316.48	3164.77
		Our	3.11	31.08	310.83	3108.31	3.16	31.63	316.32	3163.24

attributes

$$\alpha = (\alpha_1, \alpha_2, \dots, \alpha_{N_a}) \in \{0, 1\}^{\ell_1} \times \{0, 1\}^{\ell_2} \dots \{0, 1\}^{\ell_{N_a}},$$

where $|\alpha_i| = \ell_i$ and $\ell = \sum_{i \in [N_a]} \ell_i$. We set $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_{N_a})$ and define a single-point function

$$f_{\alpha, m}(x) = \begin{cases} m, & x = \alpha, \\ 0, & x \neq \alpha. \end{cases}$$

Now, suppose an aggregation task defines a (filtering) predicate policy function

$$p(x_1, x_2, \dots, x_{N_a}).$$

Our goal is to compute $p(\alpha_1, \alpha_2, \dots, \alpha_{N_a}) \cdot m$. To this end, we first define a set $\mathcal{X}$ that contains all *accepting* attributes

$$\mathcal{X} = \{x \mid \forall x \in \{0, 1\}^\ell \text{ s.t. } p(x_1, x_2, \dots, x_{N_a}) = 1\}.$$

Now we claim $\sum_{x \in \mathcal{X}} f_{\alpha, m}(x) = p(\alpha_1, \alpha_2, \dots, \alpha_{N_a}) \cdot m$. Correctness is easy to see: if $p(\alpha) = 1$, it must have $\alpha \in \mathcal{X}$, then

$$\sum_{x \in \mathcal{X}} f_{\alpha, m}(x) = f_{\alpha, m}(\alpha) + \sum_{x \in \mathcal{X} \setminus \{\alpha\}} f_{\alpha, m}(x) = m.$$

Otherwise, we have

$$p(\alpha) = 0 \Leftrightarrow \alpha \notin \mathcal{X} \Leftrightarrow \sum_{x \in \mathcal{X}} f_{\alpha, m}(x) = 0.$$

Overall, we have

$$\sum_{x \in \mathcal{X}} f_{\alpha, m}(x) = p(\alpha_1, \alpha_2, \dots, \alpha_{N_a}) \cdot m.$$

Formal description. Recall that SDPF supports decoupling the static attributes $\alpha_1, \alpha_2, \dots, \alpha_{N_a}$ from the dynamic streaming message m . Thus, the client can encode its attributes $\alpha_1, \alpha_2, \dots, \alpha_{N_a}$ using a single-point function $f_{(\alpha_1, \alpha_2, \dots, \alpha_{N_a}), (1, r)}$ in the SDPF setup phase, and invoke the SDPF encryption algorithm to encrypt m . The servers obviously share $p(\alpha_1, \alpha_2, \dots, \alpha_{N_a}) \cdot m$. Fig. 10 shows the formal non-interactive attribute-hiding aggregation based on SDPF.

Performance. We report the performance of the attribute-hiding aggregation. Note that the accepting query set $\mathcal{X}$ is at most the size of $\mathbb{G}_{\text{in}}$. Therefore, we report the *worst* efficiency when $\mathcal{X}$ contains all the elements in $\mathbb{G}_{\text{in}}$, which corresponds to the full-domain evaluation of SDPF. In this case, Table 8

Table 8: Running time (s) with varying $\mathbb{G}_{in}$ and N_c .

N_c	#Threads	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
100	1 Thread	0.02	0.05	0.19	0.74	2.94	11.98
	2 Thread	0.01	0.03	0.09	0.37	1.57	6.18
	4 Thread	0.01	0.02	0.05	0.23	0.91	3.94
	8 Thread	<0.01	0.01	0.04	0.20	0.88	3.42
1000	1 Thread	0.12	0.45	1.88	7.33	32.07	125.13
	2 Thread	0.06	0.24	0.96	3.78	15.43	62.67
	4 Thread	0.04	0.12	0.55	2.73	10.75	41.05
	8 Thread	0.02	0.08	0.38	2.30	8.53	33.52

reports the query efficiency with different sizes of $\mathbb{G}_{in}$ and number of contributors N_c . In particular, the evaluation time grows linearly with the number N_c of clients and is exponentially with the bit size $\log |\mathbb{G}_{in}|$. Overall, the running time grows linearly with N_c . Our attribute-hiding naturally supports evaluation speedup via multi-thread optimization. We depict the trend in Fig. 11.

The query efficiency would be intolerable for large $\mathbb{G}_{in}$ in the extreme case, which would be computationally expensive to evaluate. Therefore, our attribute-hiding aggregation is practical over $\mathbb{G}_{in}$ with small and moderate sizes. Nevertheless, this already suffices for real-world applications because we can leverage quantization, approximate data structure/encoding methods [24, 28] to reduce the bit size of $\mathbb{G}_{in}$, which already suffices in a practical sense. Note the SPDF-based ahAgg supports any filtering function – it does not require the contributors to know the function during setup.

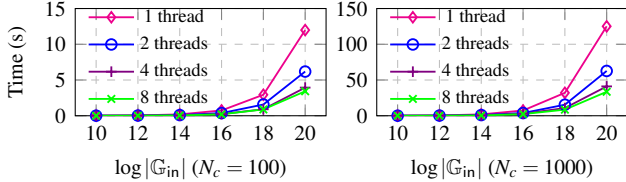


Figure 11: Evaluation speedup via multi-thread optimization.

C.2 Window-based Aggregation

Many privacy-preserving aggregation systems require window-based aggregation. In window-based aggregation, the server aggregates streaming messages over a window range $[l, r]$. Note that the SFSS-based approach already supports window-based aggregation by invoking the SFSS Enc algorithm repeatedly over the range $[l, r]$. However, this requires the servers to conduct DPF and PRF evaluation that is linear to the window width, which could be expensive for a large window range. In the following, we construct a new window-based aggregation that only requires $O(1)$ DPF and PRF calls per data stream following the method from [23].

Window-based aggregation: SFSS for point function.

Parameters: A SFSS scheme Π_{SFSS} for a function family $\mathcal{F}_{\mathbb{G}_{in}, \mathbb{G}_{out}}$ with domain $\mathbb{G}_{in}$ and range $\mathbb{G}_{out}$;

Setup($i, 1^\lambda, \hat{f} \in \mathcal{F}; \perp; \perp$):

Client i :

- 1: $(st^{(i)}, k_e^{(i)}, k_0^{(i)}, k_1^{(i)}) \leftarrow \Pi_{SFSS}.Gen(1^\lambda, \hat{f})$
- 2: Send $k_0^{(i)}$ to servers 0 and $k_1^{(i)}$ to server 1

Servers:

- 3: Server b receives and stores $k_b^{(i)}$ locally

Send($(i, st^{(i)}, k_e^{(i)}; k_0^i; k_1^i)$):

Client i :

- 1: Parse $st^{(i)} = \{ctr\}$ and $k_e^{(i)} = \{k_{kh}\}$
- 2: $c^{(i, ctr)} \leftarrow m - F(k_{kh}, ctr) + F(k_{kh}, ctr + 1)$
- 3: Update $st^{(i)} \leftarrow \{ctr + 1\}$

Servers:

- 4: Both servers receive and store $c^{(i, ctr)}$.

Aggregate($b, win = [l, r], att, k_b^{(i)}, \{c^{(i, j)}\}_{i \in [n], j \in win}$):

Server S_b :

- 1: **for** $i \in [n]$ **do**:
- 2: Let $c^{(i)} \leftarrow \sum_{j \in win} c^{(i, j)}$
- 3: $(\llbracket e^{(i)} \rrbracket_b, \llbracket k^{(i)} \rrbracket_b) \leftarrow \Pi_{SFSS}.Eval(b, k_b^{(i)}, att)$
- 4: $\llbracket s^{(i)} \rrbracket_b \leftarrow \llbracket e^{(i)} \rrbracket_b \cdot c^{(i)} + F(\llbracket k \rrbracket_b, l) - F(\llbracket k \rrbracket_b, r + 1)$
- 5: Set $\llbracket s \rrbracket_b \leftarrow \sum_{i \in [n]} \llbracket s^{(i)} \rrbracket_b$
- 6: Return $\llbracket s \rrbracket_b$

Figure 12: Window-based aggregation with bounded errors for SFSS over predicate function

Fig. 13 presents the window-based policy-hiding aggregation from SDPF. The idea is inspired by the two-server homomorphic stream encryption from [23], which is also used in Vizard. To encrypt the j -th message, the client will use the masks for encrypting the j -th and $(j + 1)$ -th streaming messages, such that once the streaming ciphertexts between $[l, r]$ are aggregated, all intermediate masks cancelled out so that only the l -th mask and $(r + 1)$ -th mask remain in the aggregated ciphertext. Note that ciphertext aggregation only requires simple addition over $\mathbb{G}_{out}$, and the servers need to compute and take off two masks for window-based aggregation.

Window-based aggregation: SFSS for predicate functions.

We can use the same double-key mechanism for the SFSS over predicate functions. In particular, the client uses the masks derived from the KH-PRF for the ctr and $ctr + 1$ to encrypt the ctr -th streaming message. For an aggregation task over a window $[l, r]$, the servers first aggregate over the range by summing up the streaming ciphertexts. By the definition, the only remaining masks are $F(k_{kh}, l)$ and $F(k_{kh}, r + 1)$. The server only needs to take off two masks during window-based aggregation to share the underlying aggregated value.

Parameters: A SDPF scheme Π_{SDPF} for a function family $\mathcal{F}_{\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}}}$ with domain $\mathbb{G}_{\text{in}}$ and range $\mathbb{G}_{\text{out}}$;

Setup($1^\lambda, \hat{f}_{\alpha, (\beta, r)} \in \mathcal{F}; \perp; \perp$):

Client i :

- 1: $r \xleftarrow{\$} \mathbb{F}$, $\text{ctr} = 1$.
 - 2: $(k_0^{\text{dpf}}, k_1^{\text{dpf}}) \leftarrow \Pi_{\text{DPF}}.\text{Gen}(1^\lambda, \hat{f}_{\alpha, (\beta, r)})$
 - 3: $(\llbracket \beta \rrbracket_b, \llbracket r \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{\text{dpf}}, \alpha)$ for $b \in \{0, 1\}$
 - 4: $\text{st}^{(i)} \leftarrow \{\text{ctr}\}$, $k_e \leftarrow \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}$, $k_0^{(i)} \leftarrow \{k_0^{\text{dpf}}\}$, $k_1^{(i)} \leftarrow \{k_1^{\text{dpf}}\}$
 - 5: Store $(\text{st}^{(i)}, k_e^{(i)}, k_0^{(i)}, k_1^{(i)})$ and send $k_b^{(i)}$ to server b .
- Servers:
- 6: Server b receives and stores $k_b^{(i)}$ locally.

Send($(\text{st}^{(i)}, k_e^{(i)}; k_0^{(i)}, k_1^{(i)})$):

Client i :

- 1: Parse $\text{st}^{(i)} = \{\text{ctr}\}$ and $k_e = \{\llbracket r \rrbracket_0, \llbracket r \rrbracket_1\}$
- 2: $c^{(i, \text{ctr})} \leftarrow m - (F(\llbracket r \rrbracket_0, \text{ctr}) - F(-\llbracket r \rrbracket_1, \text{ctr})) \cdot \beta^{-1} + (F(\llbracket r \rrbracket_0, \text{ctr} + 1) - F(-\llbracket r \rrbracket_1, \text{ctr} + 1)) \cdot \beta^{-1}$
- 3: Update $\text{st}^{(i)} \leftarrow \{\text{ctr} + 1\}$

Servers:

- 4: Both servers receive and store $c^{(i, \text{ctr})}$.

Aggregate($b, \text{win} = [l, r], \text{att}, k_b^{(i)}, \{c^{(i, j)}\}_{i \in [n], j \in \text{win}}$):

Server S_b :

- 1: **for** $i \in [n]$ **do**:
- 2: Let $c^{(i)} \leftarrow \sum_{j \in \text{win}} c^{(i, j)}$
- 3: $(\llbracket e^{(i)} \rrbracket_b, \llbracket h^{(i)} \rrbracket_b) \leftarrow \Pi_{\text{DPF}}.\text{Eval}(b, k_b^{(i)}, \text{att})$
- 4: Compute $\llbracket s^{(i)} \rrbracket_b \leftarrow \llbracket e^{(i)} \rrbracket_b \cdot c^{(i)} + (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, l) - (-1)^b \cdot F((-1)^b \cdot \llbracket h \rrbracket_b, r + 1)$
- 5: Set $\llbracket s \rrbracket_b \leftarrow \sum_{i \in [n]} \llbracket s^{(i)} \rrbracket_b$
- 6: Return $\llbracket s \rrbracket_b$

Figure 13: Window-based aggregation for SDPF

D Security Proof

D.1 Proof of Theorem 1

Proof. Correctness refers to §4.2. Below we prove Theorem 1. Overall, we prove that the SDPF key k_b and ciphertexts $\{c_1, c_2, \dots, c_q\}$ are pseudorandom; thus revealing no more non-trivial information. The proof is similar to the previous proof strategy for FSS [18, 39]. Since we use a DPF construction in a black-box fashion, we can directly use the existing simulation strategy [17, 18, 39] for DPF.

For any PPT adversary $\mathcal{A} = (\mathcal{A}_0, \dots, \mathcal{A}_q)$, we construct a PPT simulator $\mathcal{S} = (\mathcal{S}_0, \dots, \mathcal{S}_q)$ such that for any point function $f_{\alpha, (\beta, r)}$, any $b \in \{0, 1\}$, $\mathcal{S}$ simulates a DPF key k_b and streaming ciphertexts $\{c_j\}_{j \in [1, q]}$ from the allowed leakage function $\text{Leak}(\hat{f}, (m_1, \dots, m_j)) = (\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda, q)$. $\mathcal{S}_0(1^\lambda, b, \text{Leak}(\hat{f}))$:

- 1: Invoke $\mathcal{S}^{\text{DPF}}$ to obtain $k_b \leftarrow \mathcal{S}^{\text{DPF}}(1^\lambda, b, \text{Leak}(\hat{f}))$
- 2: Return k_b

While for $j \in [1, q]$, the simulator $\mathcal{S}_j$ works as follows: $\mathcal{S}_j(1^\lambda, b, \perp, \text{Leak}(\hat{f}, (m_1, \dots, m_j)))$:

- 1: Sample $c_j \xleftarrow{\$} \mathbb{G}_{\text{out}}$ and return (j, c_j)

We prove that the distribution of the simulated key and ciphertexts is indistinguishable from that of the original Π_{SDPF} key generation and encoding, via a sequence of hybrid games.

- **Hyb₀:** This is exactly the Π_{SDPF} construction in Fig. 7.
- **Hyb₁:** Hyb₁ only differs with Hyb₀ by calling $\mathcal{S}^{\text{DPF}}(1^\lambda, b, \text{Leak}(\hat{f}))$ to generate k_b^{dpf} . The adversary can only distinguish the view from Hyb₀ and Hyb₁ with negligible probability from the pseudorandomness of k_b^{dpf} ; a concrete $\mathcal{S}^{\text{DPF}}$ can be found from [18, 39].
- **Hyb₂:** Hyb₂ differs with Hyb₁ on the generation of c . In Hyb₁, we have

$$c_j \leftarrow m_j - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot F((-1)^b \cdot \llbracket r \rrbracket_{1-b}, j)),$$

while in Hyb₂, we have $k \xleftarrow{\$} \mathbb{G}_{\text{out}}$ and Hyb₂ computes

$$c_j \leftarrow m_j - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot F(k, j)).$$

The replacement is computationally indistinguishable from the pseudorandom-share property of FSS.

- **Hyb₃:** Hyb₃ differs with Hyb₂ on the generation of c . In Hyb₂, we have

$$c_j \leftarrow m_j - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot F(k, j)),$$

while in Hyb₃, we have $\mu \xleftarrow{\$} \mathbb{G}_{\text{out}}$ and Hyb₃ computes

$$c_j \leftarrow m_j - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot \mu).$$

We prove Hyb₃ and Hyb₂ are computationally indistinguishable via Claim 1.

Claim 1. *There exists a polynomial p' such that for any $(T, \epsilon_{\text{PRF}})$ -secure pseudorandom function F , then for any $b \in \{0, 1\}$ and any non-uniform PPT adversary $\mathcal{D}$ running in time $T' \leq T - p'(\lambda)$, it holds that*

$$\left| \Pr \left[\begin{array}{l} (k_b, \{c_j\}_{j \in [q]}) \leftarrow \text{Hyb}_1(1^\lambda, b) : \\ \mathcal{D}(1^\lambda, \text{st}_{\mathcal{A}}, (k_b, \{c_j\}_{j \in [q]})) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} (k_b, \{c_j\}_{j \in [q]}) \leftarrow \text{Hyb}_2(1^\lambda, b) : \\ \mathcal{D}(1^\lambda, \text{st}_{\mathcal{A}}, (k_b, \{c_j\}_{j \in [q]})) = 1 \end{array} \right] \right| \leq \text{negl}(\lambda),$$

Proof. Given an distinguisher $\mathcal{D}$ that can distinguish Hyb₂ and Hyb₃ with advantage ϵ , we construct an adversary $\mathcal{B}$ for PRF F . Note that $\mathcal{B}$ can query a PRF oracle O that is either instantiated as PRF F or a random function f .

- For each streaming encryption, $\mathcal{B}$ query $O(j)$ and receives μ .
- $\mathcal{B}$ computes the ciphertext with $c_1 \leftarrow m - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot \mu)$.
- $\mathcal{B}$ perform other computation as Hyb_2 does for key generation and streaming encryption.
- Return $(k_b, \{c_j\}_{j \in [q]})$.

Clearly, if O is instanzied by F , then $(k_b, \{c_j\}_{j \in [q]})$ generated by $\mathcal{B}$ is identically distributed as the Hyb_2 . If O is instanzied with a random function, then $(k_b, \{c_j\}_{j \in [q]})$ is identically distributed as Hyb_3 . Therefore, $\mathcal{B}$'s advantage in distinguishing the PRF from random functions equals to the advantage that $\mathcal{D}$ distinguishes Hyb_2 from Hyb_3 . Since $\mathcal{B}$ runs additional time of $p'(\lambda)$ than $\mathcal{A}$, we have that for any $T' \leq T - p'(\lambda)$, $\mathcal{A}$ distinguishes Hyb_1 and Hyb_2 is bounded by $\epsilon = \epsilon_{\text{PRF}}$. $\square$

- Hyb_4 : The difference between Hyb_4 and Hyb_3 is that Hyb_4 replace $c_j \leftarrow m - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot \mu)$ with $c_j \xleftarrow{\$} \mathbb{G}_{\text{out}}$, which is uniformly distributed over $\mathbb{G}_{\text{out}}$. Note that μ is uniformly distributed in Hyb_3 , thus $m - ((-1)^b \cdot F((-1)^b \cdot \llbracket r \rrbracket_b, j) + (-1)^{1-b} \cdot \mu)$ is uniformly distributed. This means that Hyb_3 and Hyb_4 are identically distributed. We note that Hyb_4 is exactly the ideal world.

Combining the above hybrid games completes the proof. $\square$

D.2 Proof of Theorem 2

Proof. Since we use an FSS scheme for a function family $\mathcal{F}$ in a black-box fashion in Fig. 4, we will use the existing simulation strategy for FSS directly. The simulator follows the same proof strategy for Π_{SDPF} . For any PPT adversary $\mathcal{A} = (\mathcal{A}_0, \dots, \mathcal{A}_q)$, we construct a PPT simulator $\mathcal{S} = (\mathcal{S}_0, \dots, \mathcal{S}_q)$ such that for any function $f \in \mathcal{F}$, any $b \in \{0, 1\}$, $\mathcal{S}$ simulates an FSS key k_b and streaming ciphertexts $\{c_j\}_{j \in [1, q]}$ from the allowed leakage function $\text{Leak}(\hat{f}, (m_1, \dots, m_j)) = (\mathbb{G}_{\text{in}}, \mathbb{G}_{\text{out}} \times \{0, 1\}^\lambda, j)$ for $j \in [0, q]$. $\mathcal{S}_0(1^\lambda, b, \text{Leak}(\hat{f}))$:

- 1: Invoke $\mathcal{S}^{\text{FSS}}$ to obtain $k_b \leftarrow \mathcal{S}^{\text{FSS}}(1^\lambda, b, \text{Leak}(\hat{f}))$
- 2: Return k_b

While for $j \in [1, q]$, the simulator $\mathcal{S}_j$ works as follows: $\mathcal{S}_j(1^\lambda, b, \text{Leak}(\hat{f}, (m_1, \dots, m_j)))$:

- 1: Sample $c_j \xleftarrow{\$} \mathbb{G}_{\text{out}}$ and return (j, c_j)

We show that the distribution of simulated keys and ciphertexts is indistinguishable from the distribution of Π_{SFSS} key generation and encoding, via a sequence of hybrid games.

- Hyb_0 : This is exactly the Π_{SFSS} construction in Fig. 4.
- Hyb_1 : Hyb_1 only differs with Hyb_0 by calling $\mathcal{S}_0(1^\lambda, b, \text{Leak}(\hat{f}))$ to generate k_b . The adversary

can only distinguish the view from Hyb_0 and Hyb_1 with negligible probability from the pseudorandomness of k_b .

- Hyb_2 : The difference between Hyb_1 and Hyb_2 is in the generation of c . In Hyb_1 , we have $c_j \leftarrow m + F(k_{\text{kh}}, j)$, while in Hyb_2 , $\mu \xleftarrow{\$} \mathbb{F}$ is uniformly distributed and Hyb_2 computes $c_j \leftarrow m + u$. If $\mathcal{A}$ can distinguish Hyb_1 and Hyb_2 with non-negligible probability, then we can build an adversary $\mathcal{B}$ to distinguish F from random functions with the same advantage. Thus, by the security of F , Hyb_1 and Hyb_2 are indistinguishable.
- Hyb_3 : Hyb_3 generates $c_j \leftarrow u$, where $u \xleftarrow{\$} \mathbb{G}_{\text{out}}$ is uniformly distributed. Note that Hyb_3 works exactly as $\mathcal{S}$. We note that $m + u$ and u follow the identical uniform distribution. Hence, Hyb_3 and Hyb_2 are identically distributed.

Combining the above hybrid games completes the proof. $\square$

Remark 2. It is worth mentioning that our KH-PRF-based SFSS compiler does not require the pseudorandom-share property for the underlying FSS scheme. Intuitively, the pseudorandomness of the FSS key k_b computationally hides the augmented predicate function within its function family, which implies that the embedded KH-PRF key k_{kh} is computationally hidden. Thus, $c_j = m_j + F(k_{\text{kh}}, j)$ is computationally indistinguishable from $c_j = m_j + F(k_{\text{kh}}, j)$ over uniformly chosen key $k_{\text{kh}} \xleftarrow{\$} \mathbb{K}_{\text{kh}}$. This is different from the proof for Theorem 1, where the pseudorandom-share property is necessary because $c_j = m_j - F(\llbracket r \rrbracket_0, j) + F(-\llbracket r \rrbracket_1, j)$, in which $F((-1)^{(1-b)} \llbracket r \rrbracket_{1-b}, j)$ is the only secret part to hide m_j as the distinguisher $\mathcal{D}$ knowing k_b can compute $F((-1)^b \cdot \llbracket r \rrbracket_b, j)$. In this case, we need the pseudorandom-share property for $\llbracket r \rrbracket_{1-b}$ as PRF F requires random keys. But again, we note that the pseudorandom-share property is essentially free to obtain following the PRG-based constructions from [17, 18].